



离散数学 II

一树2

周旻
清华大学软件学院
软件工程与系统研究所

2026年4月16日

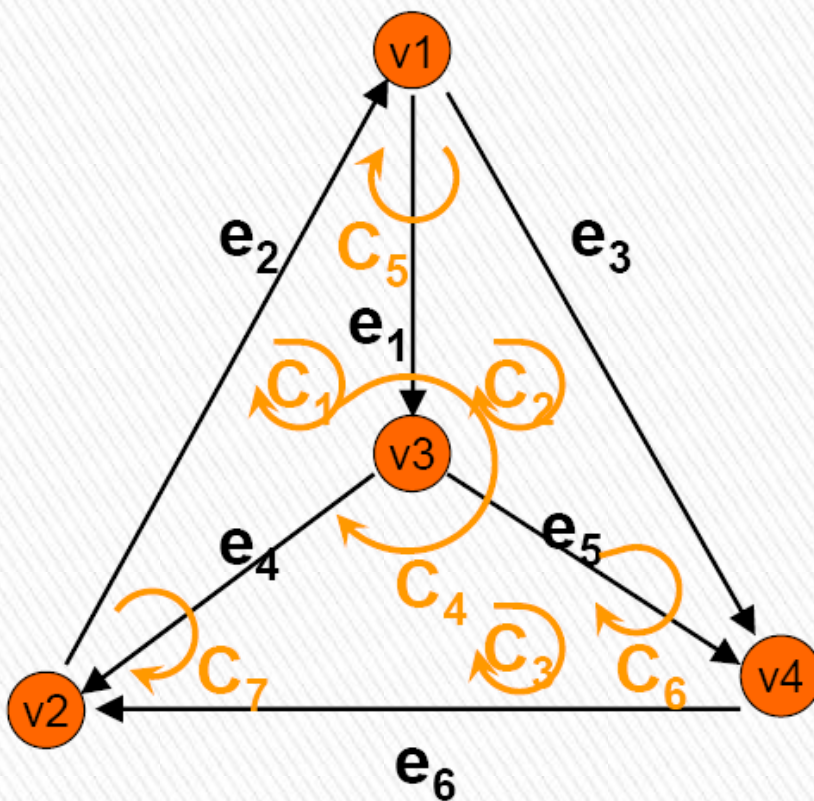
第三章 树

- 树的有关定义
- 基本关联矩阵及其性质
- 支撑树的计数
- 回路矩阵与割集矩阵
- 最短树
- 支撑树的生成
- Huffman树

回路矩阵与割集矩阵

回路矩阵

研究回路矩阵在网络、特别是电路网络中有着广泛应用。



回路矩阵

定义3.4.1 完全回路矩阵

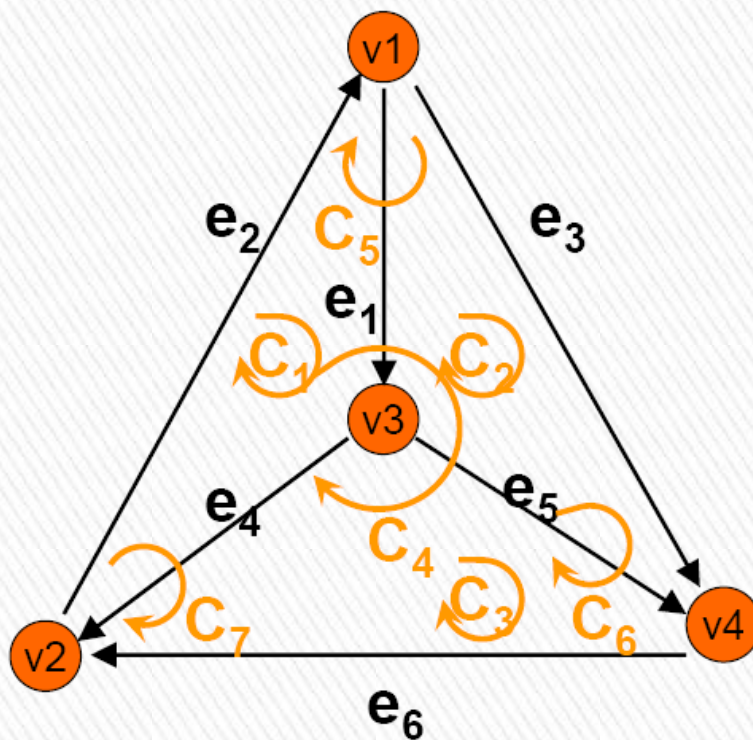
有向连通图 G 的全部初级回路构成的矩阵，称为 G 的完全回路矩阵，记为 C_e 。它的元素为：

$$c_{ij} = \begin{cases} 1 & e_j \in C_i \text{ 且与 } C_i \text{ 的方向一致} \\ -1 & e_j \in C_i \text{ 且与 } C_i \text{ 的方向相反} \\ 0 & \text{其他} \end{cases}$$

回路矩阵

完全回路矩阵可能包含多少个回路？

- 最多可能包含 $2^{m-n+1}-1$
- 是否独立呢？



回路矩阵

例：求右图的完全回路矩阵。

$$C_1 = v_1 v_2 v_3 v_1, C_2 = v_1 v_2 v_4 v_1, C_3 = v_1 v_2 v_3 v_4 v_1,$$

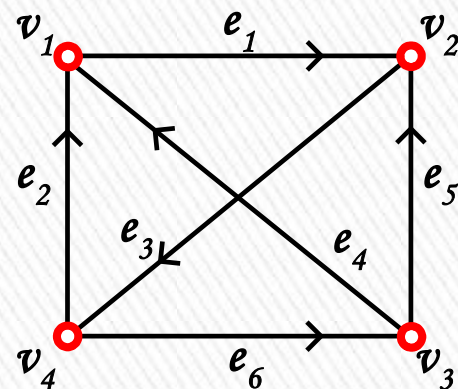
$$C_4 = v_1 v_2 v_4 v_3 v_1, C_5 = v_1 v_3 v_2 v_4 v_1, C_6 = v_1 v_3 v_4 v_1,$$

$$C_7 = v_2 v_3 v_4 v_2$$

G 的完全回路矩阵是：

$$C_e = \begin{matrix} C_1 \\ C_2 \\ C_3 \\ C_4 \\ C_5 \\ C_6 \\ C_7 \end{matrix} \begin{bmatrix} 1 & 0 & 0 & 1 & -1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ 0 & 0 & -1 & 0 & -1 & -1 \end{bmatrix}$$

$e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6$



$$C_1 \oplus C_2 = \{e_4, e_5, e_3, e_2\} = C_5$$

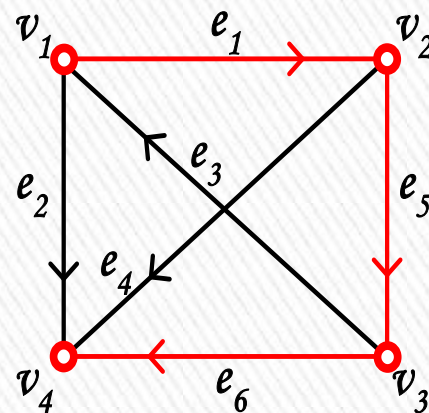
基本回路矩阵

定义3.4.2. 当有向图 $G = \langle V, E \rangle$ 的生成树 T 确定以后, 每条余树边 e 所对应的回路称为**基本回路**, 该回路的方向与 e 的方向一致。由全部基本回路构成的矩阵称为 G 的**基本回路矩阵**, 记为 C_f 。

例: 取 $T = \{e_1, e_5, e_6\}$, 则基本回路矩阵为:

$$C_f = \begin{bmatrix} -1 & 1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & -1 \end{bmatrix} \begin{matrix} T + e_2 \\ T + e_3 \\ T + e_4 \end{matrix}$$

$e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6$



基本回路矩阵

例：取 $T = \{e_1, e_5, e_6\}$ ，则基本回路矩阵为：

$$C_f = \begin{bmatrix} -1 & 1 & 0 & 0 & -1 & -1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 & -1 \end{bmatrix} \begin{matrix} T + e_2 \\ T + e_3 \\ T + e_4 \end{matrix}$$

e_1

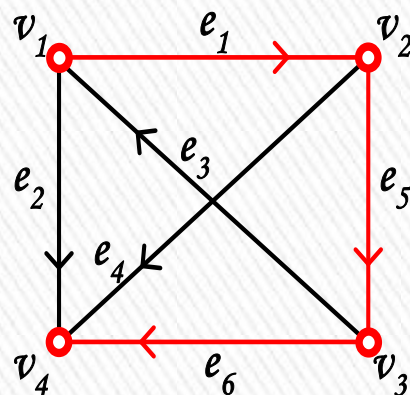
e_2

e_3

e_4

e_5

e_6



若将 C_f 的行列进行交换，使 T 的边放在后，余树边放在前，且次序与它所构成的回路一致，就可以写成分块矩阵形式。

$$C_f = \begin{bmatrix} 1 & 0 & 0 & -1 & -1 & -1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & -1 & -1 \end{bmatrix}$$

e_2

e_3

e_4

e_1

e_5

e_6

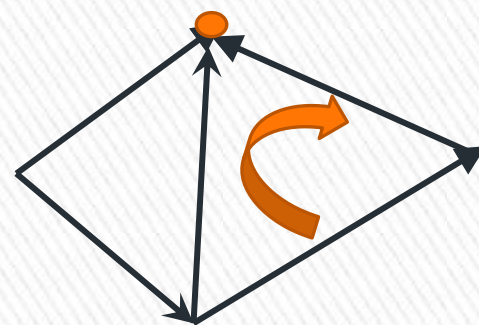
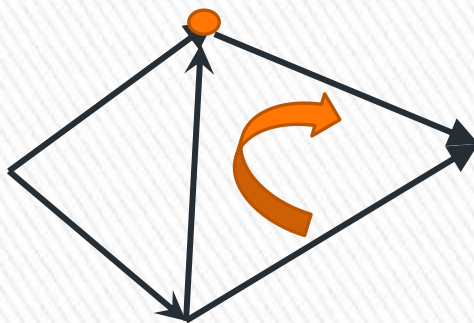
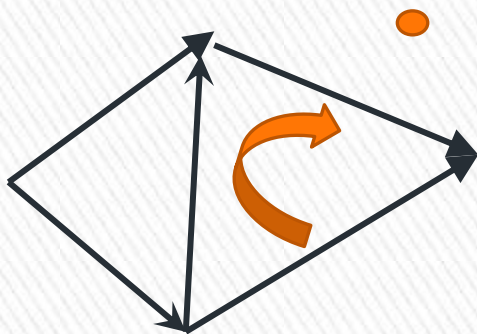
即 $C_f = [I \quad C_{f_{12}}]$ ，其中 $C_{f_{12}}$ 是 T 边所对应的子阵

回路矩阵的性质

定理3.4.1 有向连通图 $G = \langle V, E \rangle$ 的关联矩阵 B 和完全回路矩阵 C_e 的边次序一致时, 恒有 $BC_e^T = 0$ 。

证明: 设 $D = BC_e^T$, 则 $d_{ij} = \sum_{k=1}^m b_{ik}c_{jk}$, 其中

$$b_{ik} = \begin{cases} 1 & v_i \xrightarrow{e_k} \\ -1 & v_i \xleftarrow{e_k} \\ 0 & \text{其他.} \end{cases} \quad c_{jk} = \begin{cases} 1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向一致} \\ -1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向相反} \\ 0 & e_k \notin C_j. \end{cases}$$



回路矩阵的性质

定理3.4.1 有向连通图 $G = \langle V, E \rangle$ 的关联矩阵 B 和完全回路矩阵 C_e 的边次序一致时, 恒有 $BC_e^T = 0$ 。

证明: 设 $D = BC_e^T$, 则 $d_{ij} = \sum_{k=1}^m b_{ik}c_{jk}$, 其中

$$b_{ik} = \begin{cases} 1 & v_i \xrightarrow{e_k} \\ -1 & v_i \xleftarrow{e_k} \\ 0 & \text{其他.} \end{cases} \quad c_{jk} = \begin{cases} 1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向一致} \\ -1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向相反} \\ 0 & e_k \notin C_j. \end{cases}$$

回路 C_j 与点 v_i 的位置只有2种可能:

(1) C_j 不过 v_i , 则与 v_i 关联的任一边都不是 C_j 中的边

$$\therefore b_{ik} \neq 0, c_{jk} = 0 \quad \therefore d_{ij} = 0$$

(2) C_j 过 v_i , 则 C_j 必过与 v_i 关联的2条边 e_p 和 e_q

若 e_p, e_q 在 C_j 中方向一致, 则 $c_{jp} \neq 0, c_{jq} \neq 0$ 且同号

$$\text{此时, 对 } v_i \text{ 而言, } b_{ip} \text{ 与 } b_{iq} \text{ 反号} \quad \therefore d_{ij} = 0$$

若 e_p, e_q 在 C_j 中反向, 对 v_i 而言, 它们是同进同出 $\therefore d_{ij} = 0$

由 d_{ij} 的任意性, 定理得证

回路矩阵的性质

定理3.4.2 有向连通图 $G = (V, E)$ 的完全回路矩阵 C_e 的秩是 $m - n + 1$

证明:

- 由于基本回路矩阵 C_f 是完全回路矩阵 C_e 的子阵且 $\text{ran } C_f = m - n + 1$, 故 $\text{ran } C_e \geq m - n + 1$
- 现证 $\text{ran } C_e \leq m - n + 1$
- Sylvester定理指出, 两个矩阵 $A_{n \times s}$, $B_{s \times m}$, 如果 $AB = 0$, 则 $\text{ran } A + \text{ran } B \leq s$
- 因此由定理3.4.1 $BC_e^T = 0$, 得到 $\text{ran } B + \text{ran } C_e \leq m$, 即 $\text{ran } C_e \leq m - n + 1$ (关联矩阵 B)

回路矩阵的性质

定义3.4.3 由连通图 G 中 $m-n+1$ 个互相独立的回路组成的矩阵 C ，称为 G 的回路矩阵。

性质：

- (1) 基本回路矩阵 C_f 是回路矩阵
- (2) $BC^T = 0$ (其中 B, C 的边次序一致)
- (3) $C = PC_f$ (其中 P 是非奇异的方阵， C 与 C_f 的边次序一致)

证明：略

回路矩阵的性质

定理3.4.3 连通图 G 的回路矩阵 C 的任一 $m-n+1$ 阶子阵行列式非0当且仅当这些列对应于 G 的某一棵余树。(余树与其回路矩阵之间的关系)

证明：充分性

假设支撑树 T ，写出其基本回路矩阵 $C_f=[I, C_{f12}]$

对于任意的回路矩阵 C ，因为 $C=PC_f=[P, PC_{f12}]$ ，容易知道前 $m-n+1$ 阶子阵为 P ， P 非奇异，所以行列式非0

回路矩阵的性质

定理3.4.3 连通图 G 的回路矩阵 C 的任一 $m-n+1$ 阶子阵行列式非0当且仅当这些列对应于 G 的某一棵余树。(余树与其回路矩阵之间的关系)

证明：必要性

假设 $C=[C_{11}, C_{12}]$ ，且前 $m-n+1$ 阶子阵(C_{11})行列式非0，要证明这些列对应余树，只需证明 C_{12} 对应了支撑树。

反证法，假设 C_{12} 对应的不是树，则一定有回路，则至少存在1行，在 C_{11} 全是0，在 C_{12} 的存在非0。

此时调整 C 的行，改写成 $C' = \begin{bmatrix} C'_{11} & C'_{12} \\ 0 & C'_{22} \end{bmatrix}$

则 $\det(C_{11})=0$ ，矛盾

回路矩阵的性质

定理3.4.4 若有向连通图 $G = \langle V, E \rangle$ 的基本关联矩阵 B_k 和基本回路矩阵 C_f 的边次序一致，并设

$$C_f = [I, C_{f_{12}}], \quad B_k = [B_{11}, B_{12}]$$

则 $C_{f_{12}} = -B_{11}^T B_{12}^{-T}$

证明：

$$0 = B_k C_f^T = [B_{11}, B_{12}] \begin{bmatrix} I \\ C_{f_{12}}^T \end{bmatrix} = B_{11} + B_{12} C_{f_{12}}^T$$

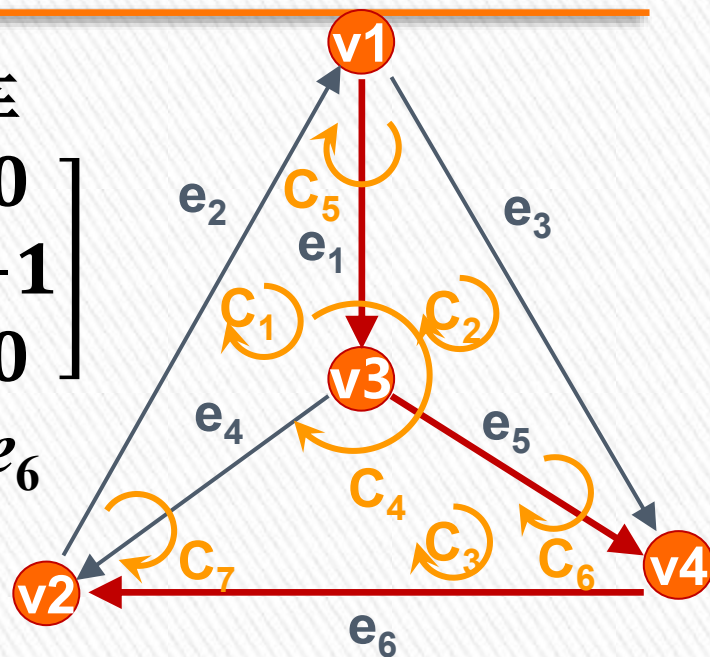
(可用来求基本回路矩阵 C_f)

回路矩阵的性质

例3.4.3 已知图3.11基本关联矩阵

$$B_4 = \begin{bmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 \\ -1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

$e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6$



其中 e_1, e_5, e_6 所对应的矩阵行列式非零，求基本回路矩阵 C_f

回路矩阵的性质

解：由 e_1, e_5, e_6 可构成 G 的一棵树，对 B 进行列变换，得到

$$B_4 = \begin{bmatrix} -1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & -1 \\ 0 & 0 & 1 & -1 & 1 & 0 \end{bmatrix} = (B_{11}, B_{12})$$

$e_2 \quad e_3 \quad e_4 \quad e_1 \quad e_5 \quad e_6$

余数边

树枝边

$$\text{其中 } B_{11} = \begin{bmatrix} -1 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 1 \end{bmatrix}, \quad B_{12} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ -1 & 1 & 0 \end{bmatrix}, \quad B_{12}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$C_{f_{12}} = -B_{11}^T B_{12}^{-T}$$

书上有误

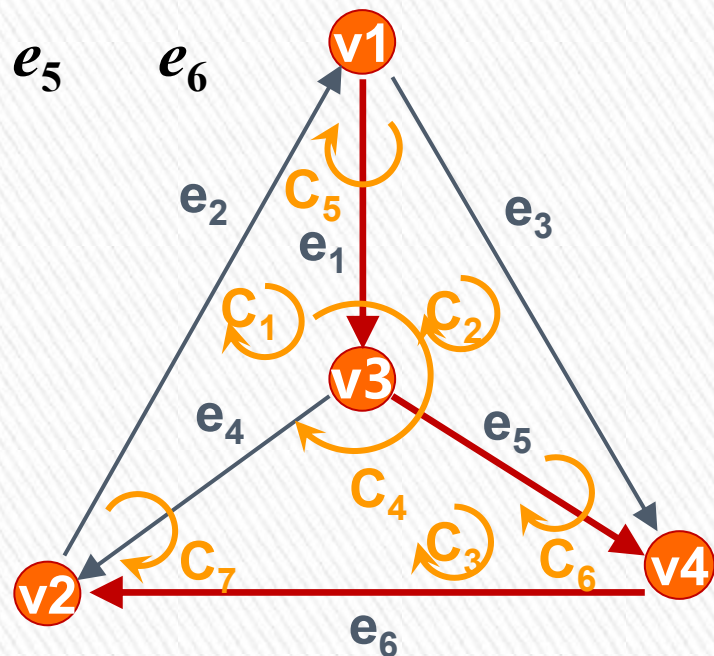
回路矩阵的性质

$$\text{因此 } C_{f_{12}} = -B_{11}^T B_{12}^{-T} = \begin{bmatrix} 1 & 1 & 1 \\ -1 & -1 & 0 \\ 0 & -1 & -1 \end{bmatrix}$$

即

$$C_f = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & -1 & -1 & 0 \\ 0 & 0 & 1 & 0 & -1 & -1 \end{bmatrix}$$

$e_2 \quad e_3 \quad e_4 \quad e_1 \quad e_5 \quad e_6$



第三章 树

- 树的有关定义
- 基本关联矩阵及其性质
- 支撑树的计数
- 回路矩阵与割集矩阵
- 最短树
- 支撑树的生成
- Huffman树

割集矩阵及其性质

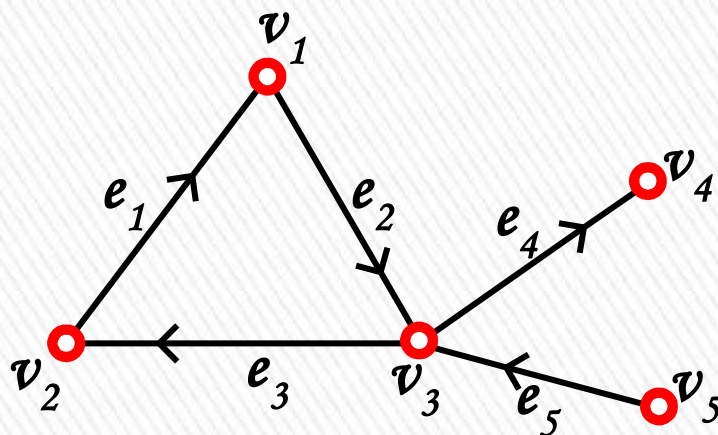
定义3.4.4 边割集

设 S 是图 $G = \langle V, E \rangle$ 的边子集, 若

(1) $G' = \langle V, E - S \rangle$ 比 G 的连通分支数多1

(2) 对 $\forall S' \subset S$, G 与 $G'' = \langle V, E - S' \rangle$ 的连通分支数相同
则称 S 为 G 的一个(边)割集

有向割集: 对有向图来说, 给割集 S 一个方向 ($\forall e \in S$, e 或与 S 同向, 或与 S 反向)



割集矩阵及其性质

定义3.4.5 完全割集矩阵

有向连通图 G 的全部割集组成的矩阵，称为完全割集矩阵，记为 S_e 。其中

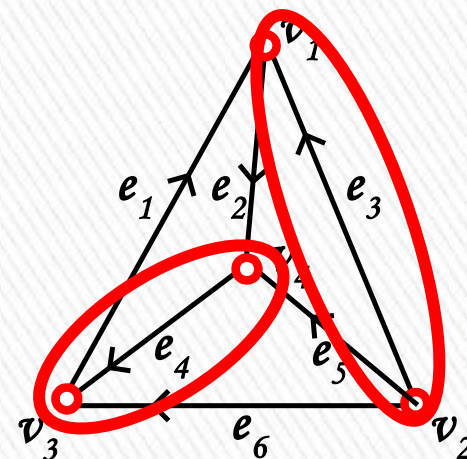
$$s_{ij} = \begin{cases} 1 & e_j \in S_i \text{ 且与 } S_i \text{ 的方向一致} \\ -1 & e_j \in S_i \text{ 且与 } S_i \text{ 的方向相反} \\ 0 & \text{其他.} \end{cases}$$

割集矩阵及其性质

例：求右图的完全割集矩阵。

$$S_e = \begin{matrix} S_1 \\ S_2 \\ S_3 \\ S_4 \\ S_5 \\ S_6 \\ S_7 \end{matrix} \begin{bmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & -1 & 1 & 0 \\ 1 & -1 & 0 & 0 & -1 & -1 \\ 0 & 1 & -1 & -1 & 0 & -1 \\ 1 & 0 & 1 & -1 & 1 & 0 \end{bmatrix}$$

$e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \quad e_6$



$$S_1 = \{e_1, e_2, e_3\}$$

$$S_2 = \{e_1, e_4, e_6\}$$

$$S_3 = \{e_3, e_5, e_6\}$$

$$S_4 = \{e_2, e_4, e_5\}$$

$$S_5 = \{e_1, e_2, e_5, e_6\}$$

$$S_6 = \{e_2, e_3, e_4, e_6\}$$

$$S_7 = \{e_1, e_3, e_4, e_5\}$$

这些割集不一定独立，如： $S_6 = S_1 \oplus S_2$

割集矩阵及其性质

- 割集将连通图的点分成两个连通分支，其点数分别为 i 和 $n - i$, $1 \leq i \leq n - 1$

故 G 最多有 $\frac{1}{2}(2^n - 2) = 2^{n-1} - 1$ 个不同的割集

- 这些割集不一定独立

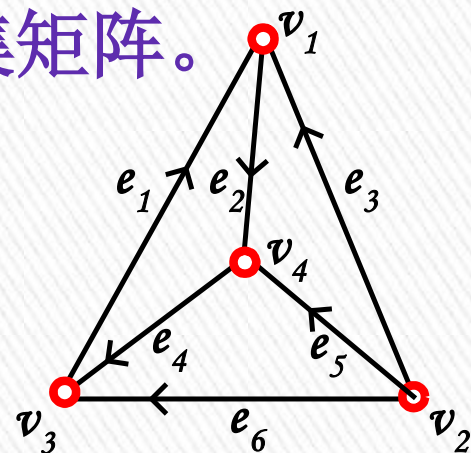
- **定义3.4.6:** 设 T 是连通图 G 的一棵树, e_i 是 T 的一条边, 对应 e_i 存在 G 的割集 S_i , S_i 只包括一条树上的边及某些余树边, 且与 e_i 的方向一致。这时称 S_i 为 G 的对应树 T 的一个基本割集

- **定义3.4.7:** 给定有向连通图 G 的一棵树, 则由全部基本割集组成的矩阵称为基本割集矩阵, 记为 S_f

割集矩阵及其性质

例：取 $T = \{e_2, e_4, e_5\}$ ，求其基本割集矩阵。

$$S_f = \begin{matrix} & \begin{matrix} S_1 \\ S_2 \\ S_3 \end{matrix} & \begin{bmatrix} -1 & 1 & -1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \\ & \begin{matrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{matrix} \end{matrix}$$



$$\begin{aligned} S_1 &= \{e_1, e_2, e_3\} \quad \checkmark \\ S_2 &= \{e_1, e_4, e_6\} \quad \checkmark \\ S_3 &= \{e_3, e_5, e_6\} \quad \checkmark \\ S_4 &= \{e_2, e_4, e_5\}, \\ S_5 &= \{e_1, e_2, e_5, e_6\}, \\ S_6 &= \{e_2, e_3, e_4, e_6\}, \\ S_7 &= \{e_1, e_3, e_4, e_5\} \end{aligned}$$

割集矩阵及其性质

- 若在本基本割集矩阵中把余树边对应的列放在前，树的边对应的列放在后，且与割集次序一致，则有

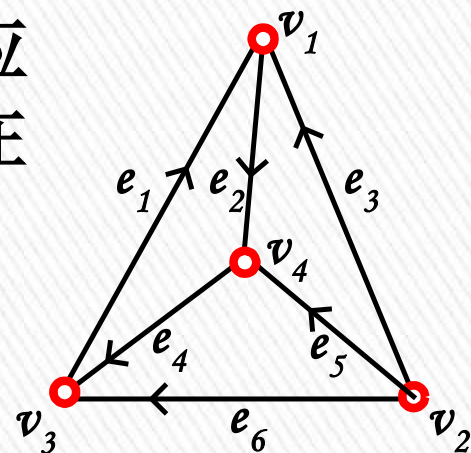
$$S_f = \begin{matrix} S_1 \\ S_2 \\ S_3 \end{matrix} \begin{bmatrix} -1 & -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{matrix} e_1 & e_3 & e_6 & e_2 & e_4 & e_5 \end{matrix}$$

$$= [S_{f_{11}}, I]$$

其中 I (单位矩阵)对应一棵树

显然 $\text{ran}(S_f) = n - 1$



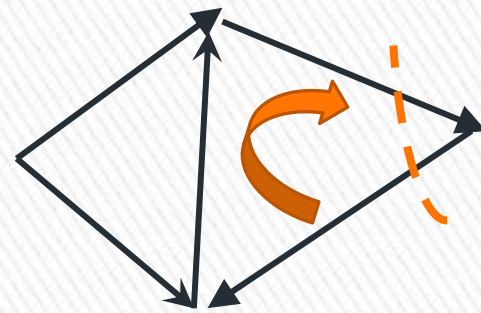
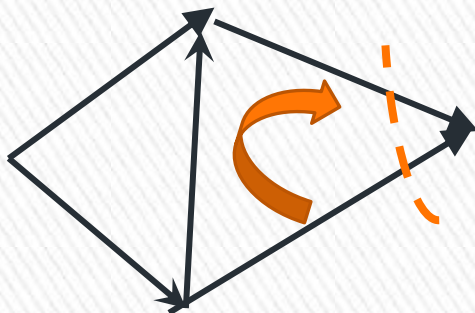
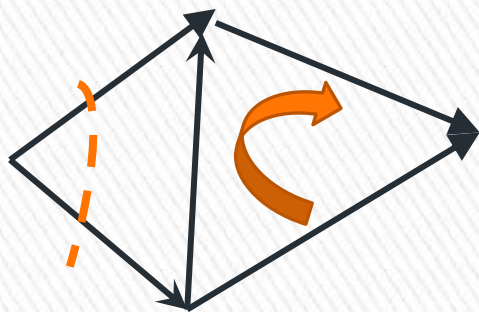
$$\begin{aligned} S_1 &= \{e_1, e_2, e_3\} \quad \checkmark \\ S_2 &= \{e_1, e_4, e_6\} \quad \checkmark \\ S_3 &= \{e_3, e_5, e_6\} \quad \checkmark \\ S_4 &= \{e_2, e_4, e_5\} \\ S_5 &= \{e_1, e_2, e_5, e_6\} \\ S_6 &= \{e_2, e_3, e_4, e_6\} \\ S_7 &= \{e_1, e_3, e_4, e_5\} \end{aligned}$$

割集矩阵的性质

定理3.4.5 当有向连通图 G 的完全回路矩阵 C_e 和完全割集矩阵 S_e 的边次序一致时, 有 $S_e C_e^T = 0$

证明: 设 $D = S_e C_e^T$, 则 $d_{ij} = \sum_{k=1}^m S_{ik} C_{jk}$, 其中 S_{ik} 是第 i 个割集中的第 k 条边, C_{jk} 是第 j 个回路中的第 k 条边

$$S_{ik} = \begin{cases} 1 & e_k \in S_i \text{ 且与 } S_i \text{ 的方向一致} \\ -1 & e_k \in S_i \text{ 且与 } S_i \text{ 的方向相反} \\ 0 & \text{其他.} \end{cases} \quad C_{jk} = \begin{cases} 1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向一致} \\ -1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向相反} \\ 0 & \text{其他.} \end{cases}$$



割集矩阵的性质

定理3.4.5 当有向连通图 G 的完全回路矩阵 C_e 和完全割集矩阵 S_e 的边次序一致时, 有 $S_e C_e^T = 0$

证明: 设 $D = S_e C_e^T$, 则 $d_{ij} = \sum_{k=1}^m S_{ik} C_{jk}$, 其中 S_{ik} 是第 i 个割集中的第 k 条边, C_{jk} 是第 j 个回路中的第 k 条边

$$S_{ik} = \begin{cases} 1 & e_k \in S_i \text{ 且与 } S_i \text{ 的方向一致} \\ -1 & e_k \in S_i \text{ 且与 } S_i \text{ 的方向相反} \\ 0 & \text{其他.} \end{cases} \quad C_{jk} = \begin{cases} 1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向一致} \\ -1 & e_k \in C_j \text{ 且与 } C_j \text{ 的方向相反} \\ 0 & \text{其他.} \end{cases}$$

割集 S_i 与回路 C_j 的关系只有2种:

(1) S_i 与 C_j 不相交。即 C_j 中的边在 S_i 中不出现, 则 $S_{ik} = 0$ 或 $C_{jk} = 0 \therefore d_{ij} = 0$

(2) S_i 与 C_j 相交。显然它们有偶数条共同的边, 其中相邻两条边 e_p, e_q 组成一对。如果它们在 S_i 中方向一致, 则在 C_j 中方向相反。反之亦然 $\therefore d_{ij} = 0$

割集矩阵的性质

定理3.4.6 连通图 G 的完全割集矩阵 S_e 的秩是 $n - 1$

证明：显然 $\text{ran}(S_e) \geq n - 1$ ，又 $S_e C_e^T = 0$

$\therefore$ 由Sylvester定理， $\text{ran}(S_e) + \text{ran}(C_e) \leq m$ ，

而 $\text{ran}(C_e) = m - n + 1$

$\therefore \text{ran}(S_e) \leq n - 1$

$\therefore \text{ran}(S_e) = n - 1$

定义3.4.8 连通图 G 的 $n-1$ 个互相独立的割集构成的矩阵称为 G 的割集矩阵，记为 S

性质：

(1) 基本割集矩阵 S_f 是割集矩阵

(2) $SC^T = 0$ ， S 和 C 的边次序一致

(3) $S = PS_f$ ，其中 P 为非奇异方阵， S 与 S_f 的边次序一致

割集矩阵的性质

定理3.4.7 连通图 $G = \langle V, E \rangle$ 的割集矩阵 S 的任一 $n-1$ 阶子阵行列式非0，当且仅当这些列对应于 G 的某棵树

定理3.4.8 若有向连通图 $G = \langle V, E \rangle$ 中关于某棵树的基本割集矩阵 S_f 和基本回路矩阵 C_f 的边次序一致，并设

$$S_f = [S_{f_{11}}, I], C_f = [I, C_{f_{12}}]$$

则 $S_{f_{11}} = -C_{f_{12}}^T$

推论3.4.1 若连通图 $G = \langle V, E \rangle$ 的基本割集矩阵与基本关联矩阵的边次序一致时，有 $S_{f_{11}} = B_{12}^{-1} B_{11}$ (可用来求基本割集矩阵 S_f)

$$C_{f_{12}} = -B_{11}^T B_{12}^{-T}$$

最短树

最小支撑树

■ 最小支撑树（最短树）

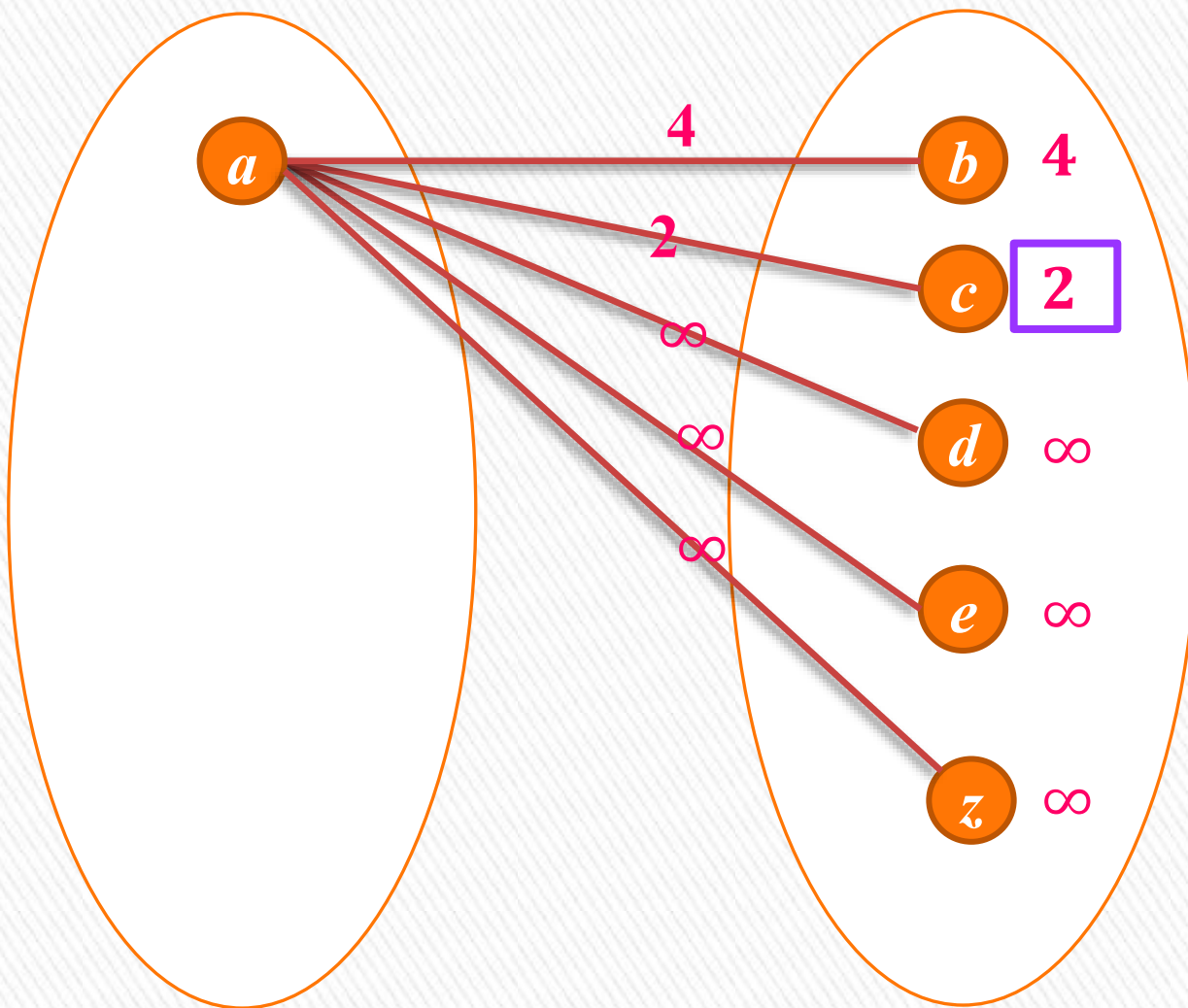
- 定义：一棵支撑树中的所有边的权之和称为该支撑树的权。具有最小权的支撑树，称为最小支撑树（Minimal Spanning Tree）。
- 实际应用很广
例如城市间的通信网络问题，如何布线，使得总的线使总长度最短。

最小支撑树的生成

连通

没有回路

权值和最小



$[a, b, c, d, e, z]$
 $c = [a, a, a, a, a, a]$

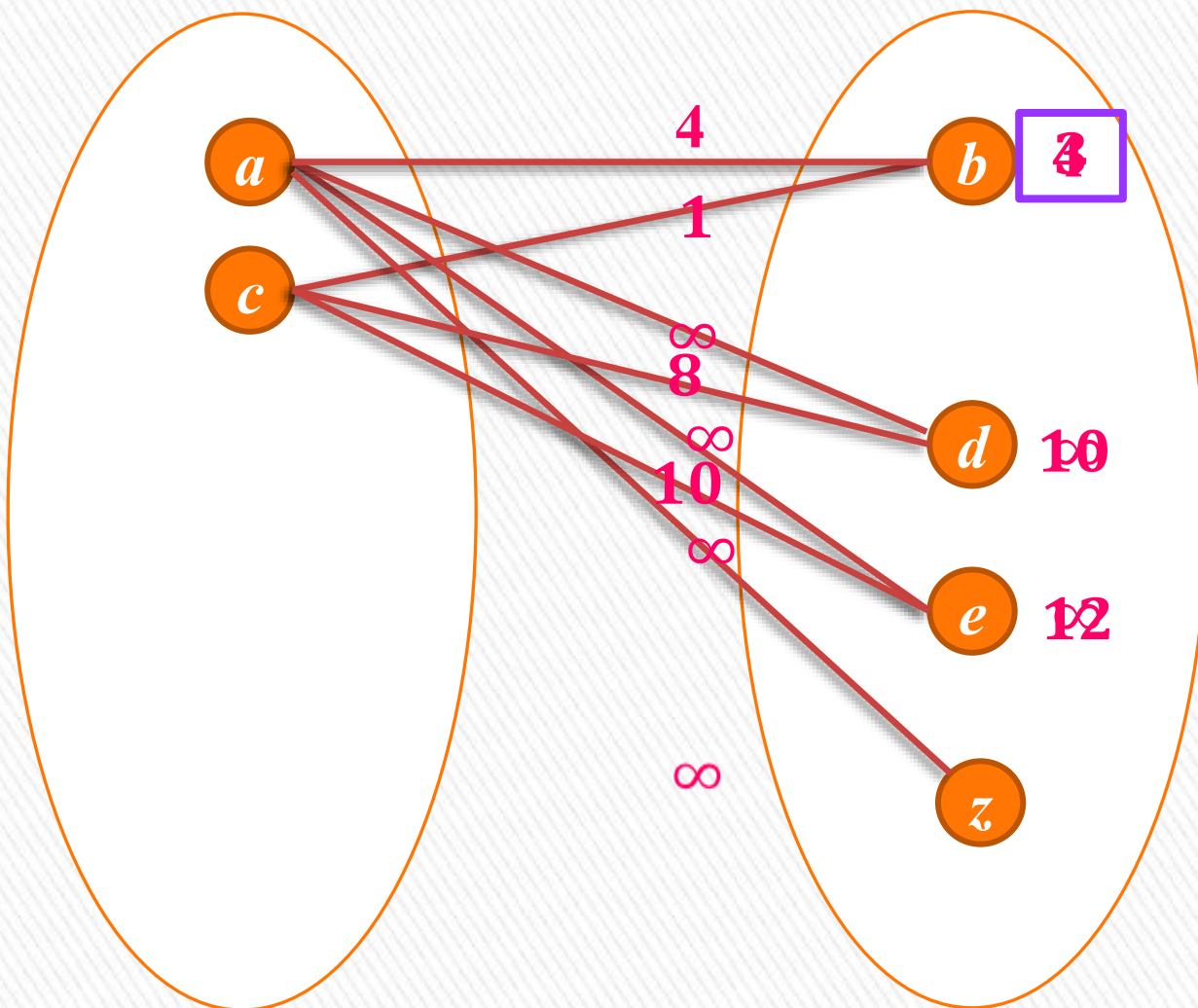
最小支撑树的生成

连通

没有回路

权值和最小

$[a, b, c, d, e, z]$
 $c = [a, a, a, a, a, a]$



最小支撑树

■ Prim算法

- 基本思想:

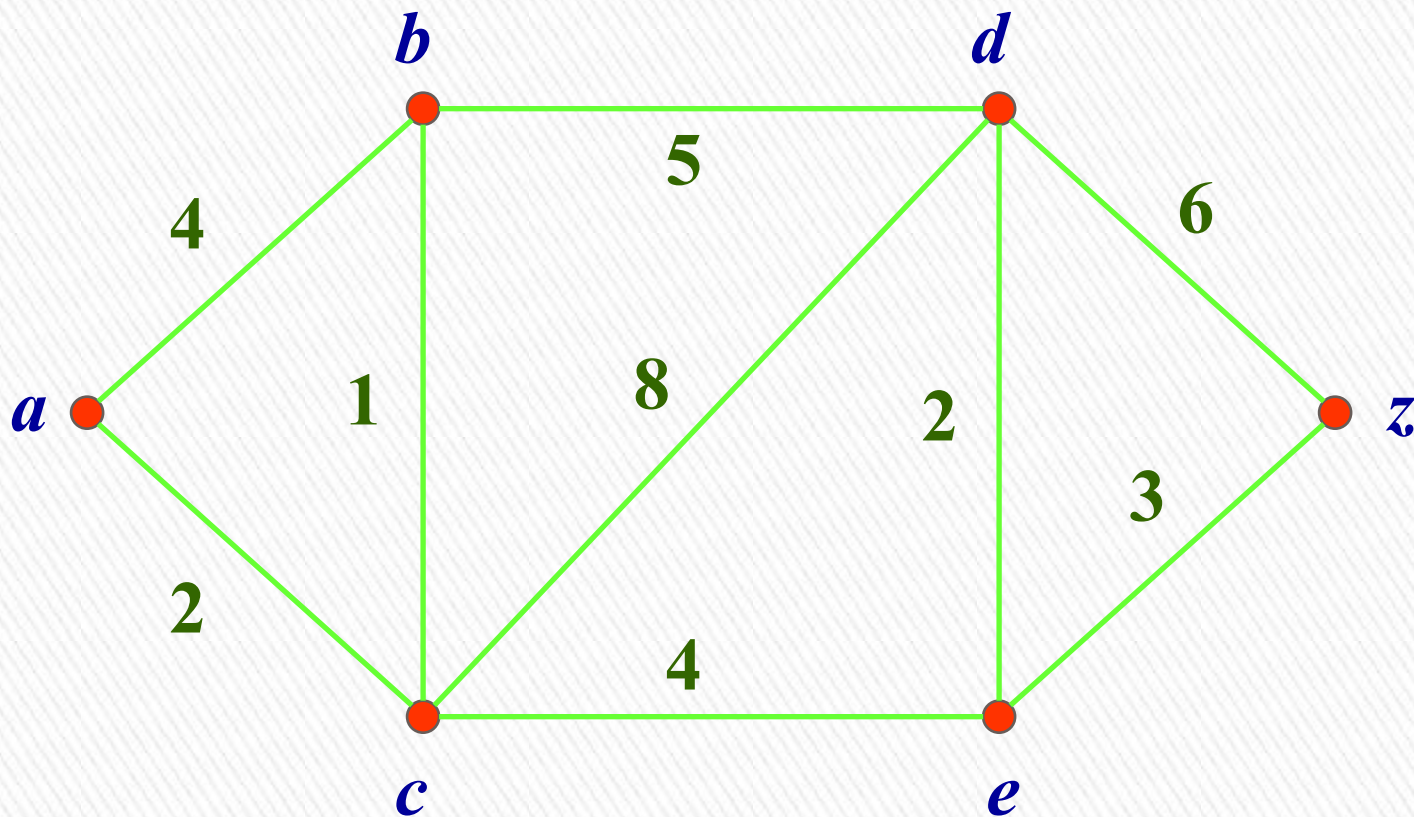
首先任选一个结点 v_0 构成集合 V' , 然后不断在 $V-V'$ 中选一条到 V' 中某点(如点 v)最短的边 (u, v) 进入树 T , 并令 $V' = V' + u$, 直至 $V' = V$

- 实现:

与Dijkstra算法类似

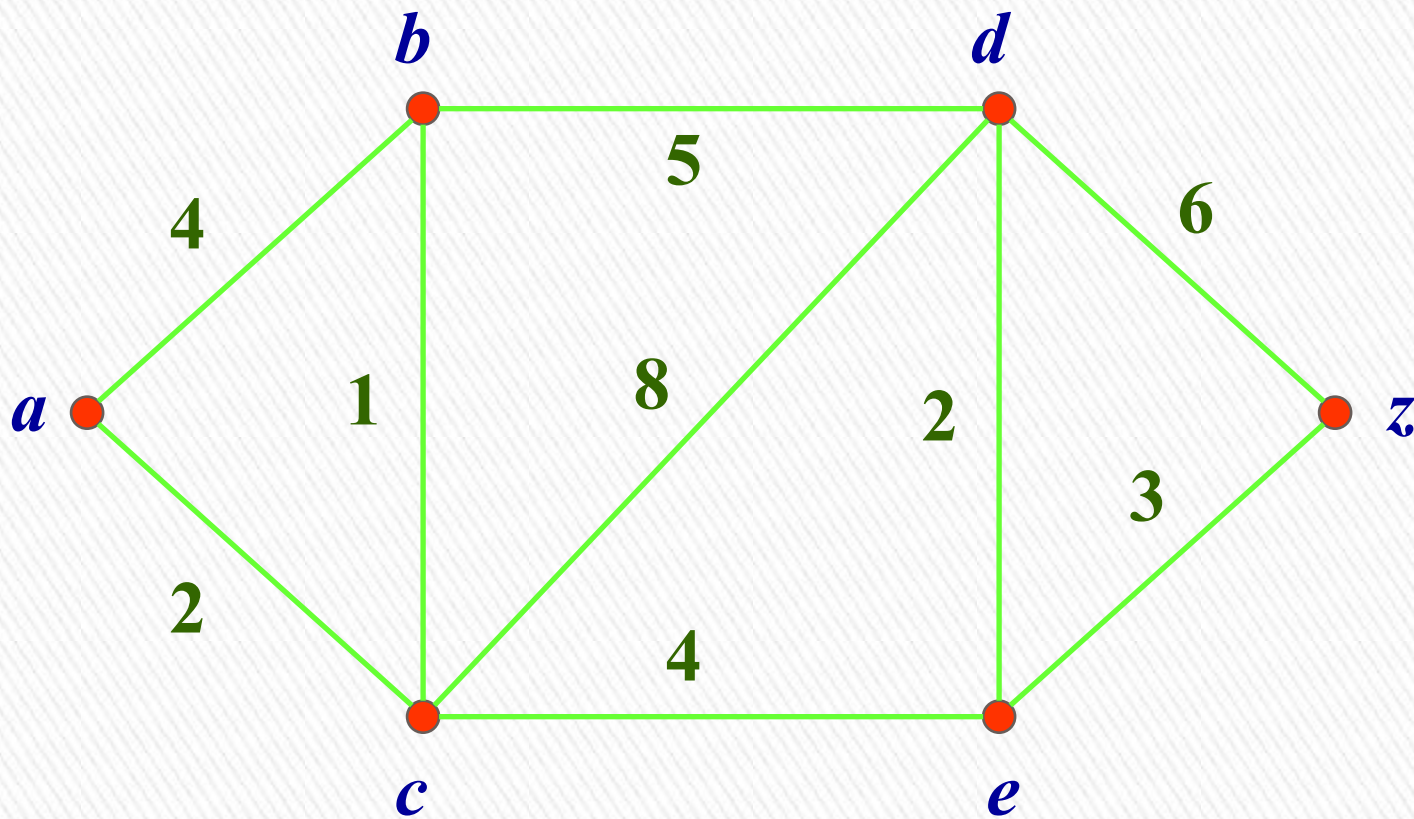
Prim实例

例：求下图最小生成树



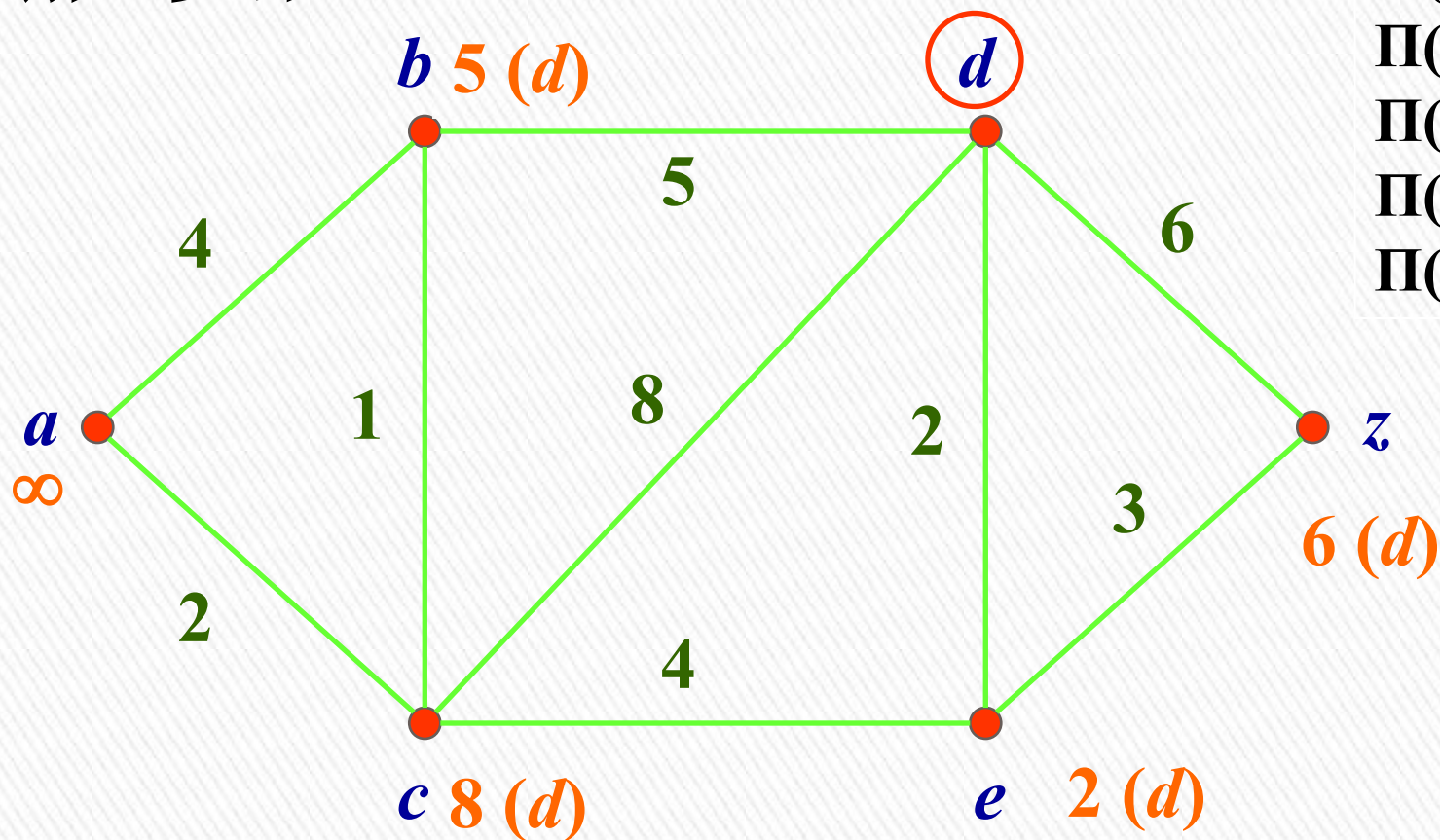
Prim实例

解：步骤1



Prim实例

解：步骤1



$$\Pi(a) = \infty$$

$$\Pi(b) = 5$$

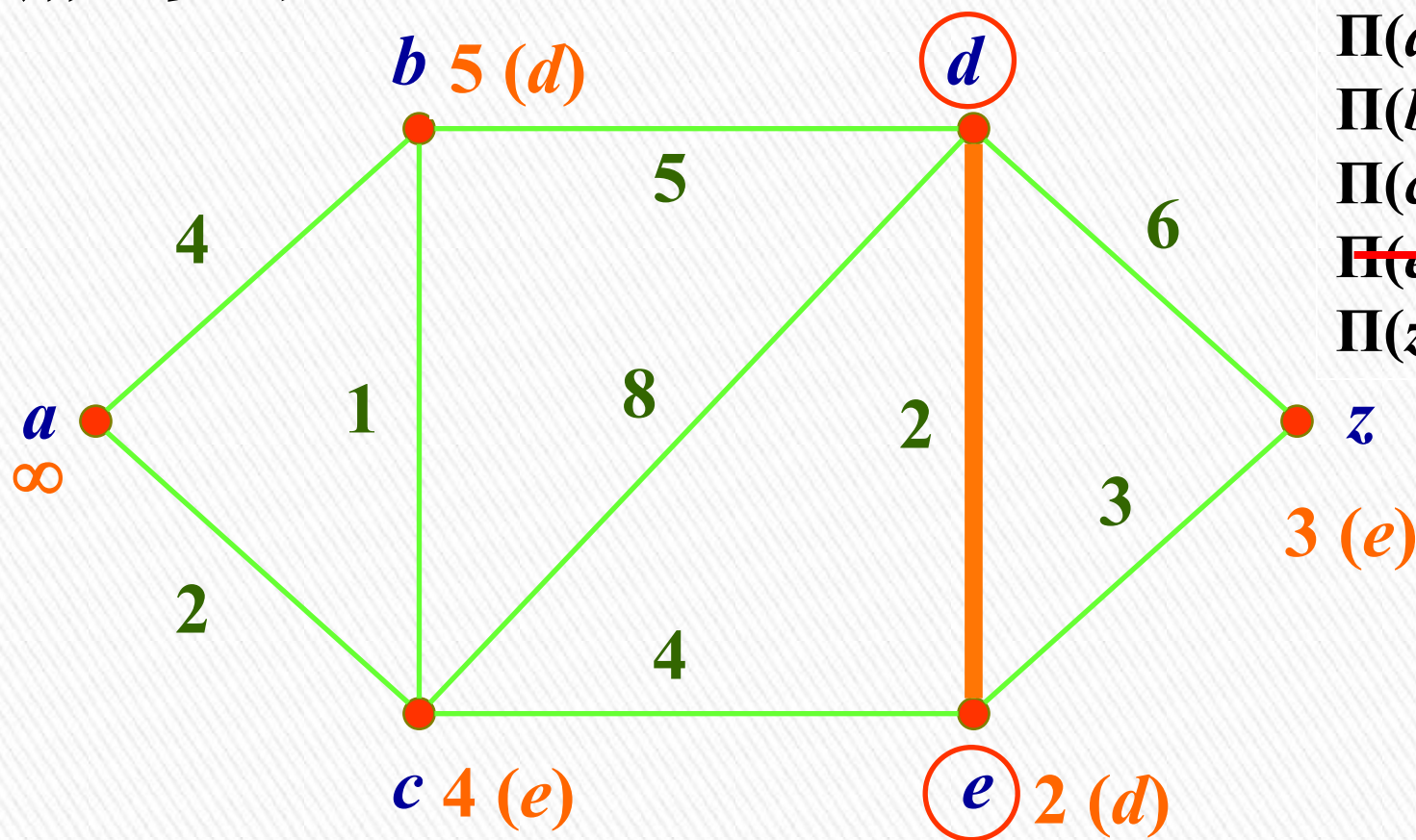
$$\Pi(c) = 8$$

$$\Pi(e) = 2$$

$$\Pi(z) = 6$$

Prim实例

解：步骤2



$$\min \Pi = 2$$

$$\Pi(a) = \infty$$

$$\Pi(b) = 5$$

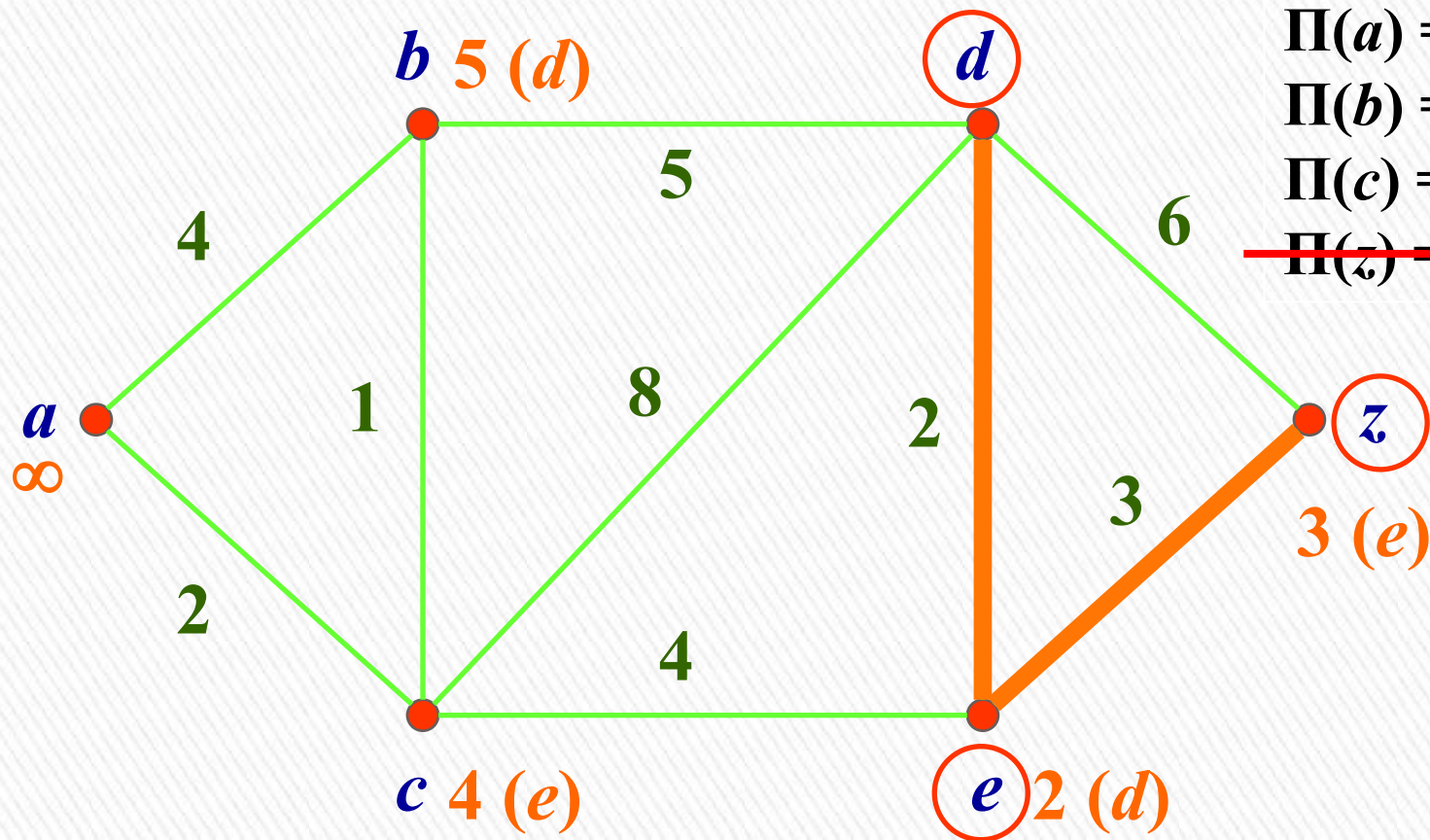
$$\Pi(c) = 8 \rightarrow 4$$

$$\Pi(e) = 2$$

$$\Pi(z) = 6 \rightarrow 3$$

Prim实例

解：步骤3



$$\min \Pi = 3$$

$$\Pi(a) = \infty$$

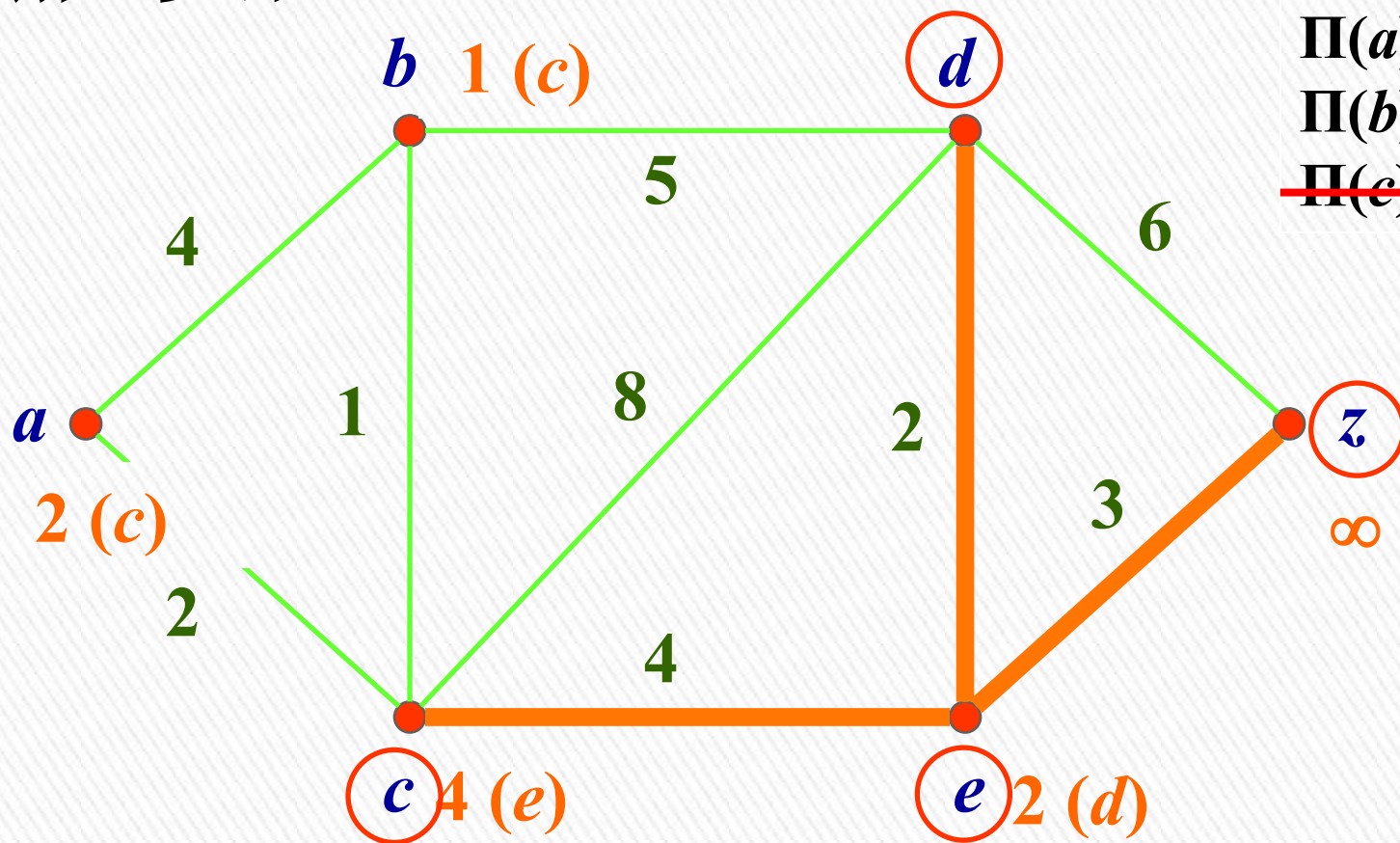
$$\Pi(b) = 5$$

$$\Pi(c) = 4$$

~~$$\Pi(z) = 3$$~~

Prim实例

解：步骤4



$$\min \Pi = 4$$

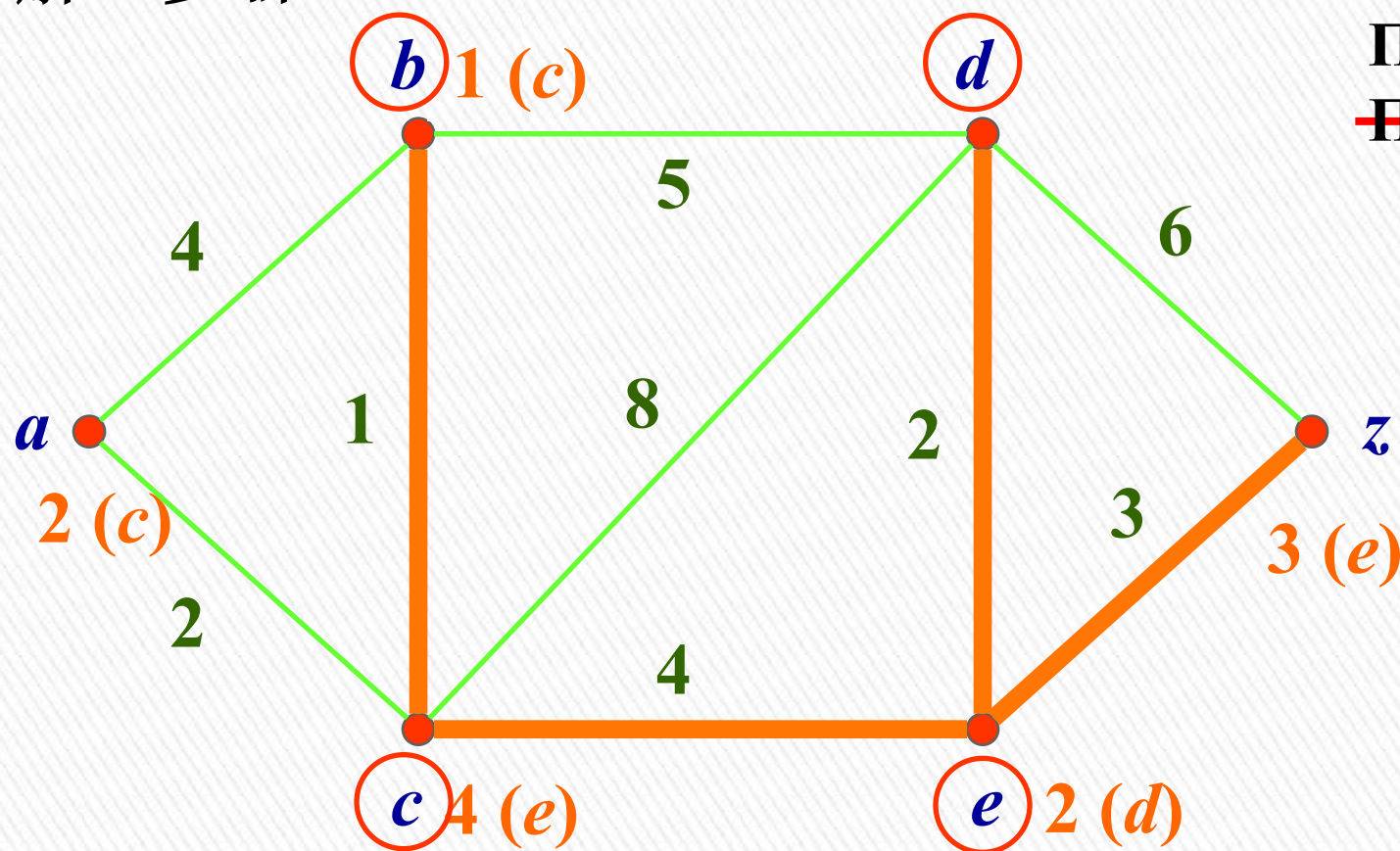
$$\Pi(a) = \infty \rightarrow 2$$

$$\Pi(b) = 5 \rightarrow 1$$

~~$$\Pi(c) = 4$$~~

Prim实例

解：步骤5



$$\min \Pi = 1$$

$$\Pi(a) = 2$$

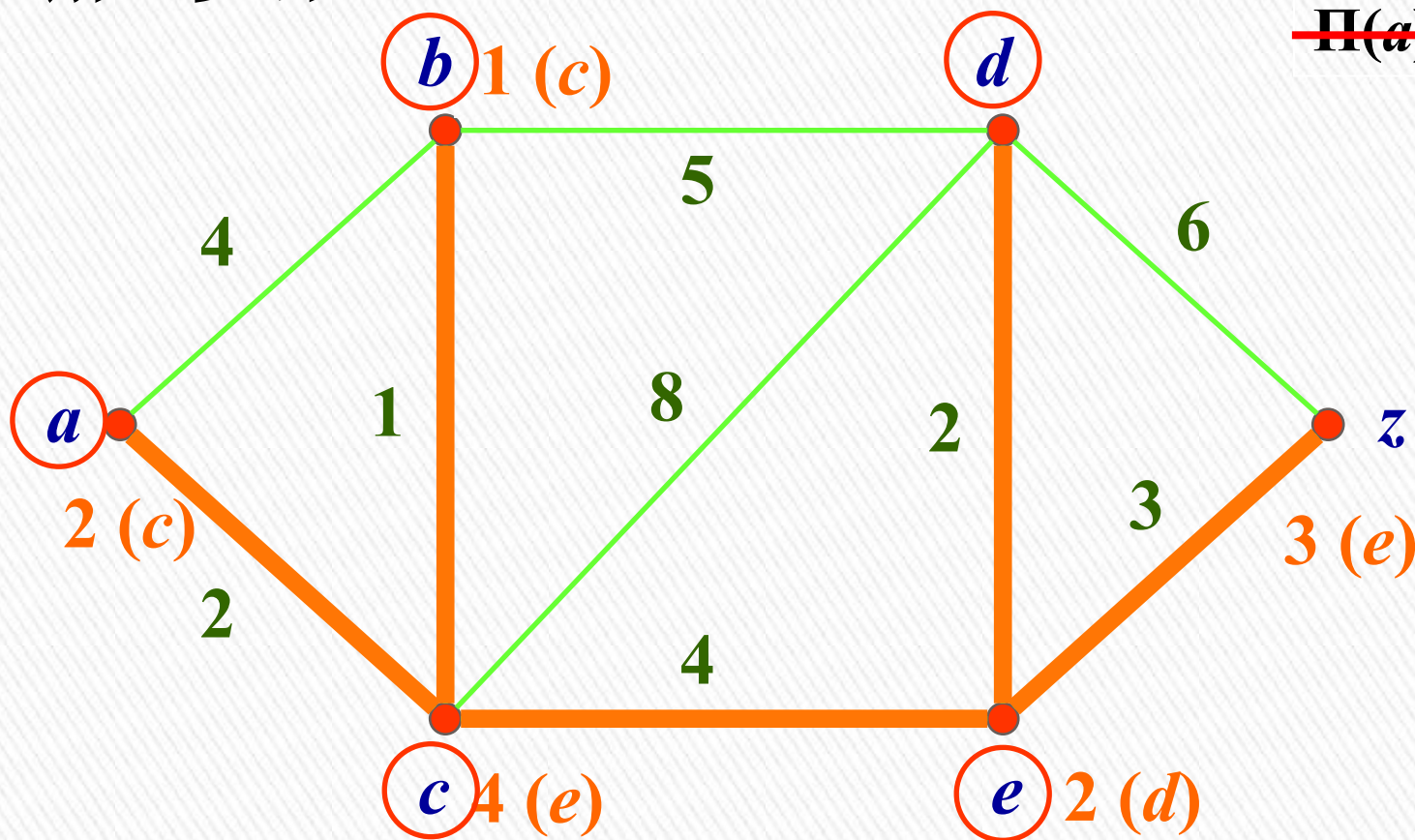
$$\text{---} \Pi(b) = 1$$

Prim实例

解：步骤5

$\min \Pi = 2$

~~$\Pi(a) = 2$~~



最小支撑树

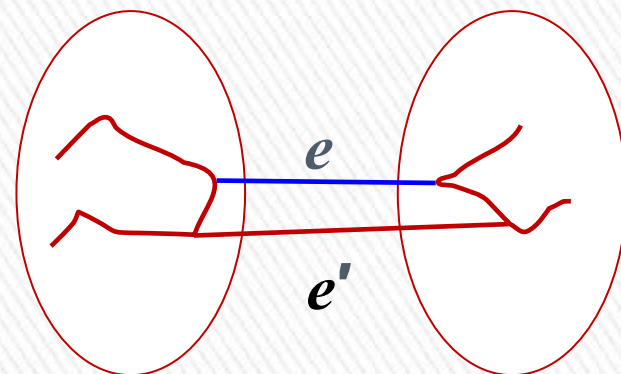
定理3.6.3 设 V' 是赋权连通图 $G = (V, E)$ 的结点真子集, e 是二端点分跨在 V' 和 $V - V'$ 的最短边, 则 G 中一定存在包含 e 的最短树 T 。

证明:

设 T_0 是 G 的一棵最短树,

如果 $e \notin T_0$, 则 $T_0 + e$ 构成唯一回路, 该回路一定包含 e 和 $e' = (u, v)$, 其中 $u \in V'$, $v \in V - V'$, 由已知条件 $w(e) \leq w(e')$, 作 $T_0 \oplus (e, e')$, 得到的仍然是最短树。

因此Prim算法得到的是最小树



最小支撑树

■ Kruskal算法

- 基本思想:

不断地往 T 中添加当前的最短边，如果此时会构成回路，那么它一定是这个回路的最长边，删之。直至最后达到 $n-1$ 条边为止。

最小支撑树

■ Kruskal算法

$T = \text{empty graph}$

$E =$ 将 m 条边按权从小到大顺序排序为: $e_1, e_2, \dots, e_m$

for $i = 1$ **to** $n - 1$

{

在 E 中按顺序取边, 直到找到不与 T 中的边构成回路的边 e_j

在 E 中删去 e_j 以及它前面的边

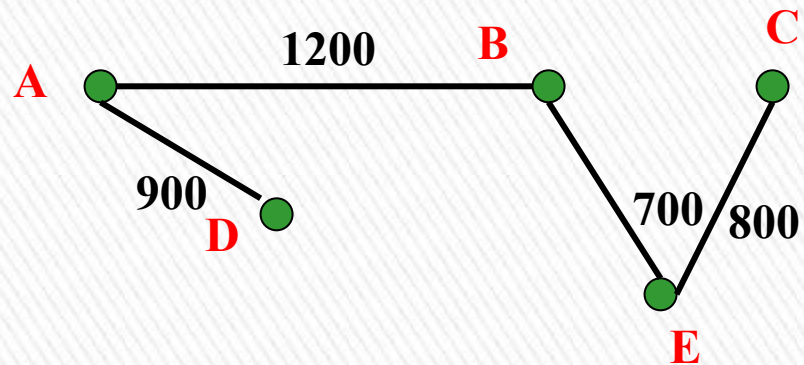
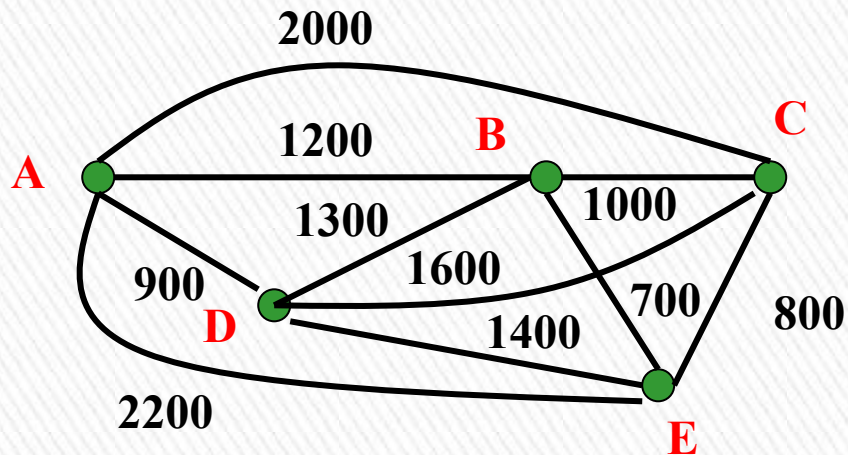
$T = T + e_j$

}

// T 是 G 的最小支撑树

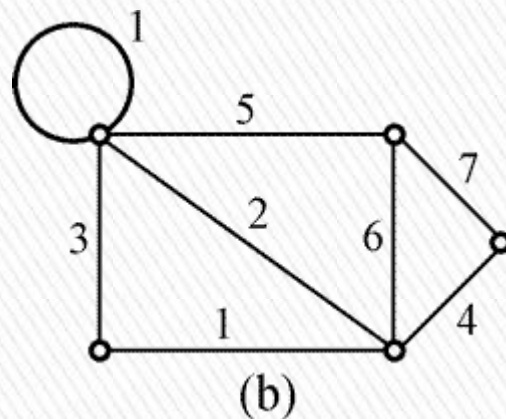
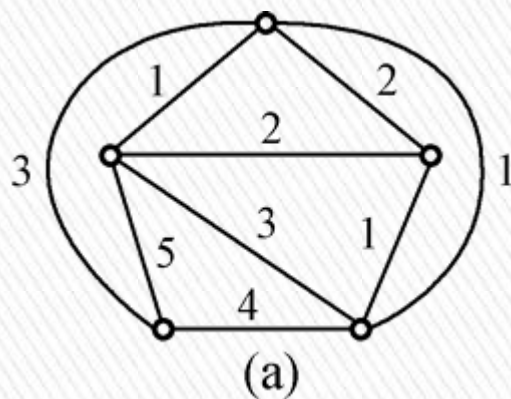
最小支撑树

■ Kruskal算法



Choice	Edge	Cost
1	BE	700
2	EC	800
3	AD	900
4	AB	1200
Total:		3600

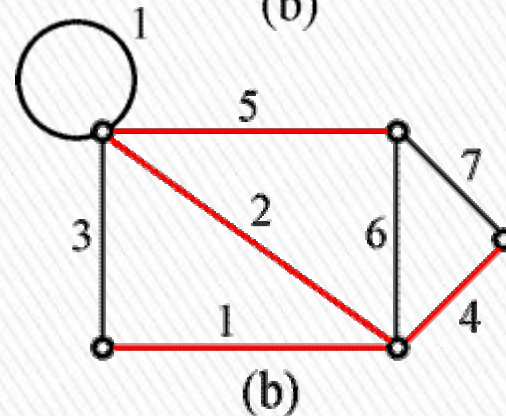
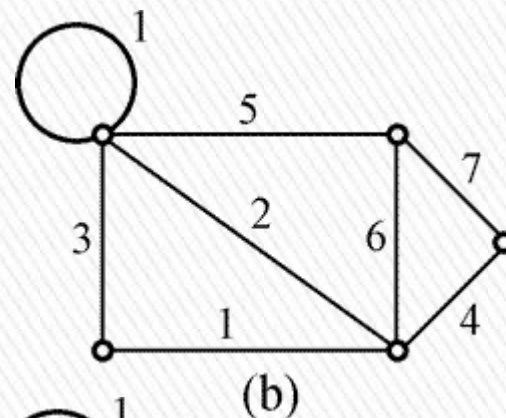
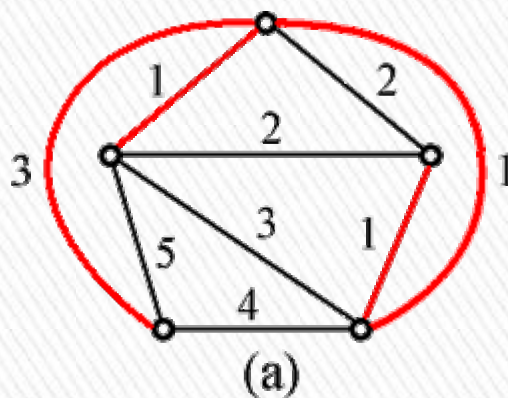
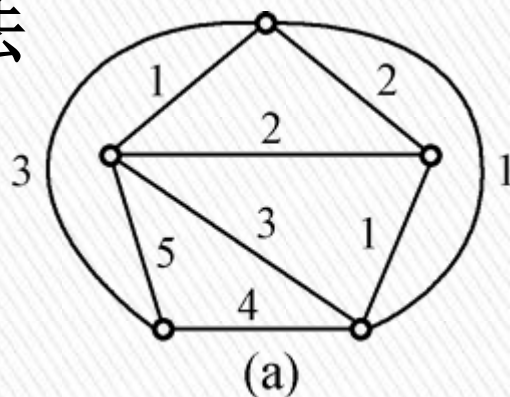
用Kruskal算法生成下图最小生成树，
图(a)的最小支撑树的权值和为 [填空1]，
图(b)的最小支撑树的权值和为 [填空2]。



最小支撑树

■ Kruskal算法

解



图(a)的最小生成树 T_1 为下图(a), $W(T_1) = 6$

图(b)的最小生成树 T_2 为下图(b), $W(T_2) = 12$

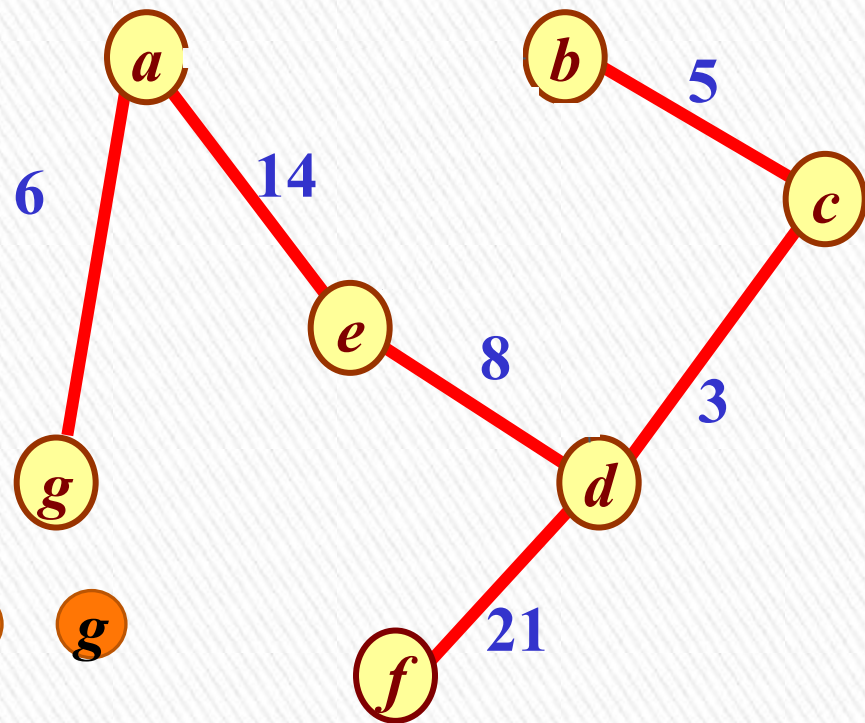
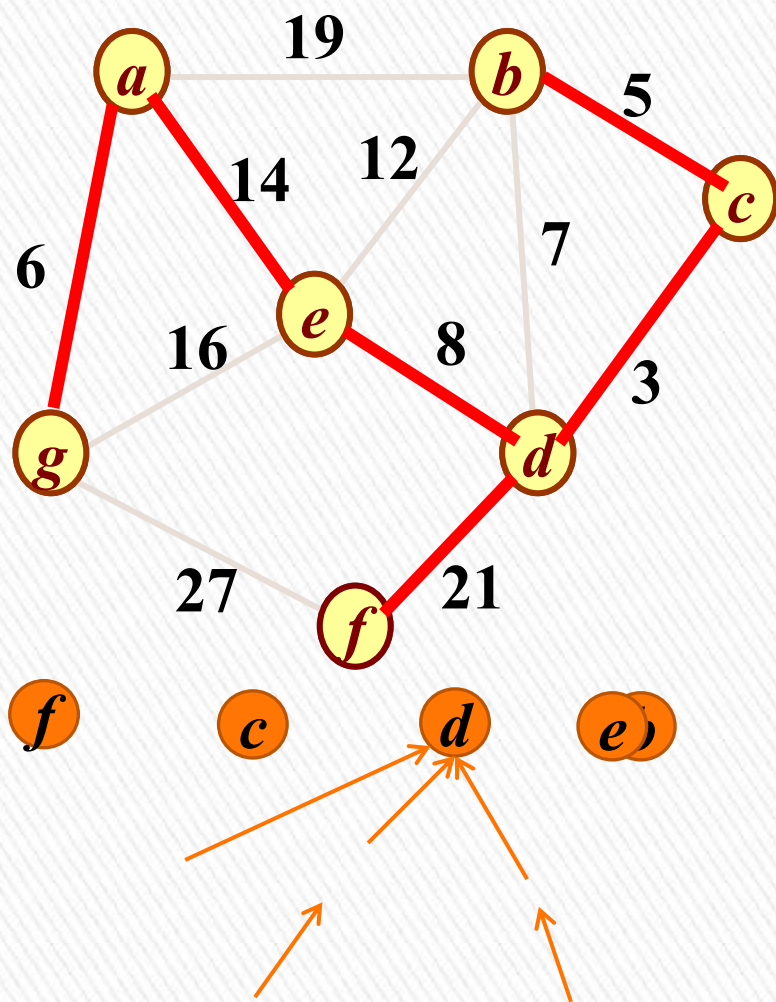
最小支撑树

■ 关键实现难点：如何判断是否形成回路

基本策略：

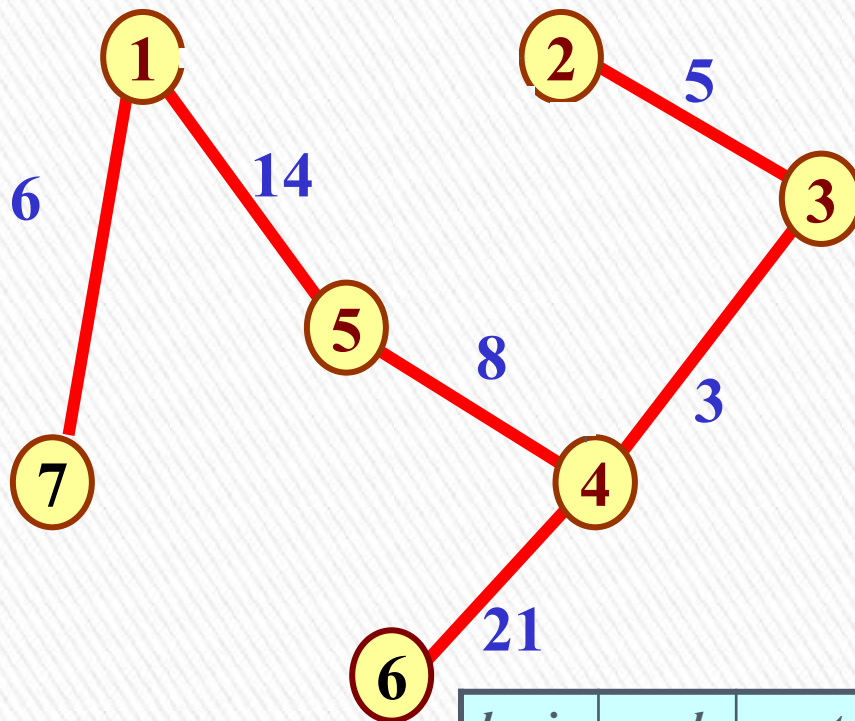
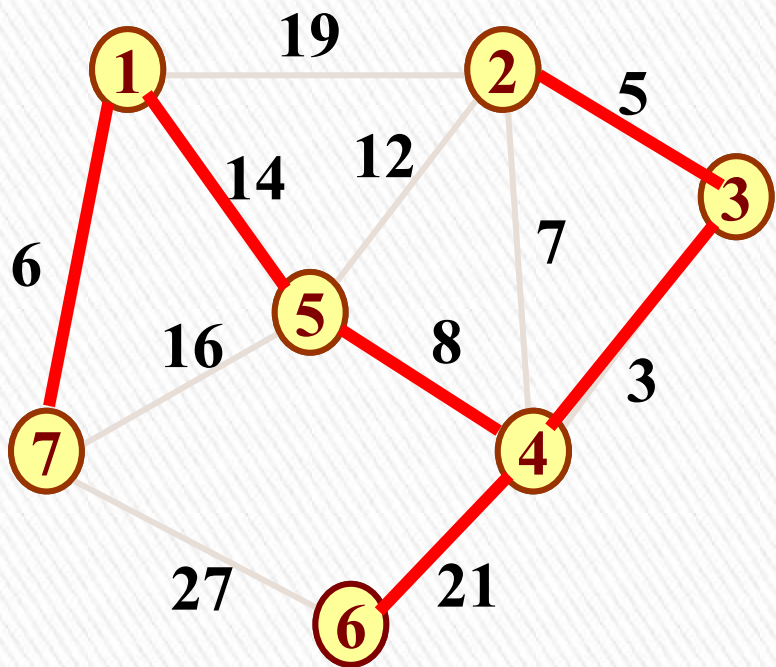
- ① 首先将 G 的 n 个顶点看成 n 个孤立的连通分支
- ② 将所有的边按权从小到大排序
- ③ 依边权递增的顺序查看每一条边 (v, w) :
 - a) 若 v 和 w 分别是当前2个不同的连通分支（树） $T1$ 和 $T2$ 中的顶点时，就用边 (v, w) 将 $T1$ 和 $T2$ 连接成一个连通分支（树），然后接着查看下一条边；
 - b) 若 v 和 w 在当前的同一个连通分支中，就直接再查看下一条边。
- ④ 重复步骤3一直进行到只剩下一个连通分支时为止。

最小支撑树



并查集算法

最小支撑树



parent[i]

1	2	3	4	5	6	7
5	3	4	5	-1	4	1
<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>

<i>begin</i>	<i>end</i>	<i>cost</i>
3	4	3
2	3	5
1	7	6
2	4	7
5	4	8

最小支撑树

```
void MiniSpanTree_Kruskal(MGraph G) {  
    //对每个顶点初始化parent数组, 结点编号[0...numVertexes-1]  
    for (i = 0; i <= G.numVertexes; i++)  
        parent[i] = -1;  
    //假设每条按权值从小到大排序  
    i = 0; numt = 0;  
    while ( (i < G.numEdges) && (numt < G.numVertexes) ) {  
        n = find_root(parent, edges[i].src);    // 查边头的根  
        m = find_root(parent, edges[i].dst);    // 查边尾的根  
        if (n != m) {                            // 此边没有形成回路  
            union(parent, n, m);                // 合并  
            numt++;  
        }  
        i++;  
    }  
}
```

最小支撑树

■ 并查集（Disjoint Sets）：

- 支持“并”和“查”的（不相交）集合
- 将集合元素放到一棵树中，用树根做代表元

```
int find_root(int *parent, int n) {  
    while (parent[n] != -1)  
        n = parent[n];  
    return n;  
}  
// 朴素的实现  
int union(int *parent, int x, int y) {  
    parent[x] = y;  
}
```

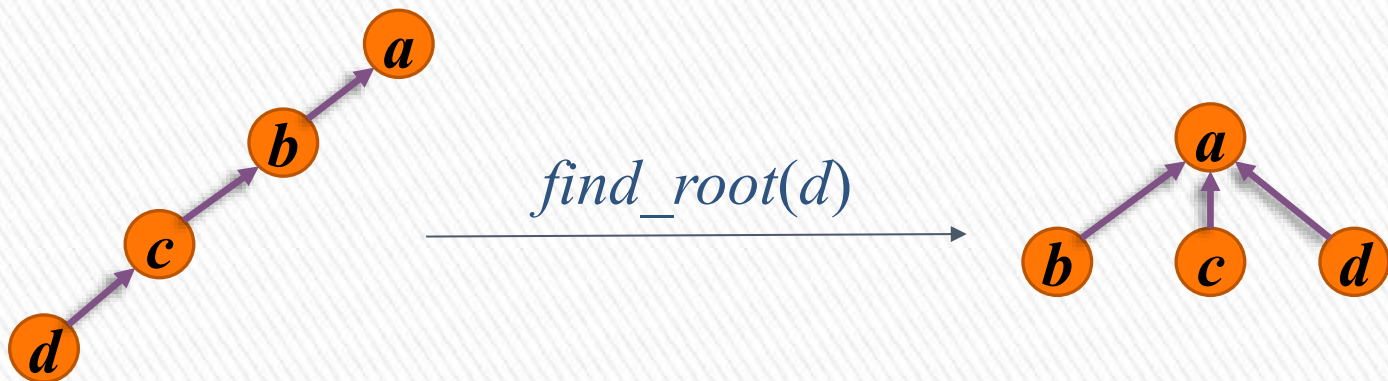
最小支撑树

■ 朴素的实现效率不佳

- 树可能退化成一条链;
- 每次“并”和“查”都需要 $O(n)$ 的时间;
- 一共 m 轮迭代, 复杂度 $O(nm)$;

■ 常用的优化方法（不考）

- 优化1（并）：小并入大 / 低并入高 $\rightarrow O(\log n)$
- 优化2（查）：路径压缩 $\rightarrow O(\log n)$



最小支撑树

■ 两个优化一起用：

- 每一次 *find / union* 的均摊时间复杂度 $O(\alpha(n))$ ，其中 α 是 Ackermann 函数的反函数，增长极慢，因此近似为常数时间；
- 如果边已经有序，则整体时间复杂度近似为 $O(p)$ ， p 是迭代次数；
- 如果需要对边排序，朴素的方法是 $O(m \log m)$ ，或者用书上的建堆的方法，复杂度是 $O(m + p \log m)$ ，两者均不小于 $O(p)$ ；

最小支撑树

- 定理：Kruskal算法找到的树是最小支撑树
- 证明：假设Kruskal算法找到的树是 T_0 ，假设 T_1 是任意一棵支撑树，讨论其差别：

边长升序	e_1	e_2	e_3	e_4	...	e_{n-1}	e_n
T_0	√	√	√	√			√
T_1	√	√		√		√	√

- 如果 $T_0 \neq T_1$ ，则最小的不同边 e_i 一定在 $T_0 - T_1$ 中，且：
 - e_i 是 T_1 的余树边， $T_1 + e_i$ 中存在唯一回路 C ；
 - C 中一定包含不小于 e_i 的边 e_j ，否则 T_0 包含 C 中所有边；
 - 此时 $T_2 = T_1 + e_i - e_j$ 仍然是一棵支撑树，且 $\omega(T_1) \geq \omega(T_2)$ ；
- 反复操作，直至 $T_n = T_0$ ，此时 $\omega(T_1) \geq \dots \geq \omega(T_n) = \omega(T_0)$
- 证明了任何一棵支撑树都不小于 T_0 。

最小支撑树

■ 两种方法均是贪心算法

- 求出的都是最优解
- Kruskal算法的计算复杂度为 $O(m + p \log m)$ ，其中 p 为迭代次数（ p 介于 n 到 m 之间）；
- Prim算法的计算复杂度为 $O(n^2)$ ；
- 稠密图应该用Prim算法，Kruskal适合于稀疏图；
- 如果利用堆等结构，算法复杂度还可以进一步降低。

支撑树的生成

(不考)

支撑树的生成

- 为方便起见，本节中用 t 表示 G 的一棵树
- 假定 t_1, t_2 是连通图 G 的两棵树， t_1 中共有 k 条边不属于 t_2 ，则称 t_1 和 t_2 的距离 $d(t_1, t_2) = k$
- 当然，此时也有 $d(t_2, t_1) = k$

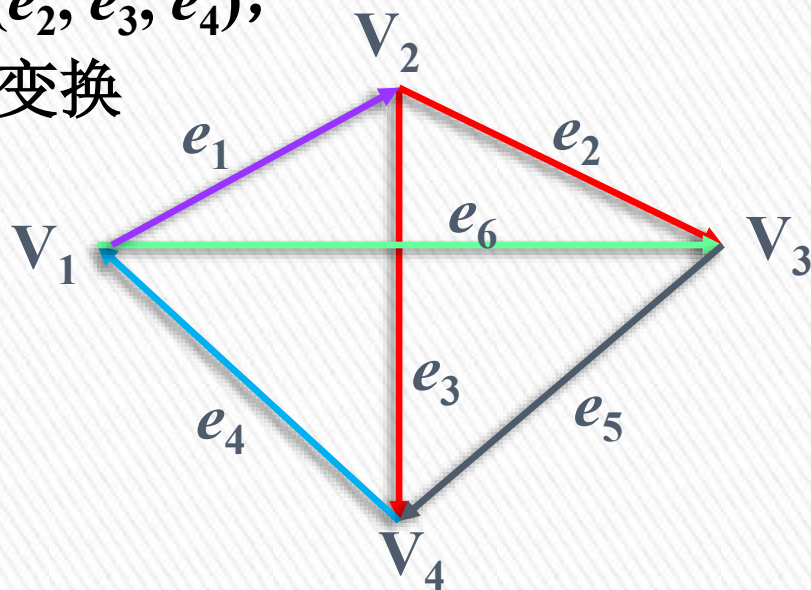
支撑树的生成

定义3.7.1 设 t_1, t_2 是连通图 G 距离为1的两棵树。

且 $t_1 - t_2 = (e)$, $t_2 - t_1 = (e')$, 则 $t_2 = t_1 \oplus (e, e')$ 称为 t_1 到 t_2 的基本树变换。

由于 $e' \notin t_1$, 因此它是 t_1 的一条余树边。 $t_1 + (e')$ 含有一条唯一回路 C , $e \in C$ 。所以 $t_1 \oplus (e, e')$ 又形成了 G 另一棵树 t_2 。

例3.7.1 右图中 $t_1 = (e_2, e_3, e_1)$, $t_2 = (e_2, e_3, e_4)$,
则 $t_2 = t_1 \oplus (e_1, e_4)$ 是 t_1 到 t_2 的基本树变换



支撑树的生成

定理3.7.1 令 t_1, t_2 是 G 中距离为1(有一条边不同)的两棵树, 且 $t_1 - t_2 = (e)$, $t_2 - t_1 = (b)$ 。则 $b \in S_e(t_1)$ 。

反之, 若 $b \in S_e(t_1)$, 则 $t_1 \oplus (e, b)$ 是树。

证明: 由 $t_1 - t_2 = (e)$, $t_2 - t_1 = (b)$ 可得 $t_1 \oplus t_2 = (e, b)$ 。

设 $t_1 = (e, a_1, a_2, \dots, a_k)$, $t_2 = (b, a_1, a_2, \dots, a_k)$ 。

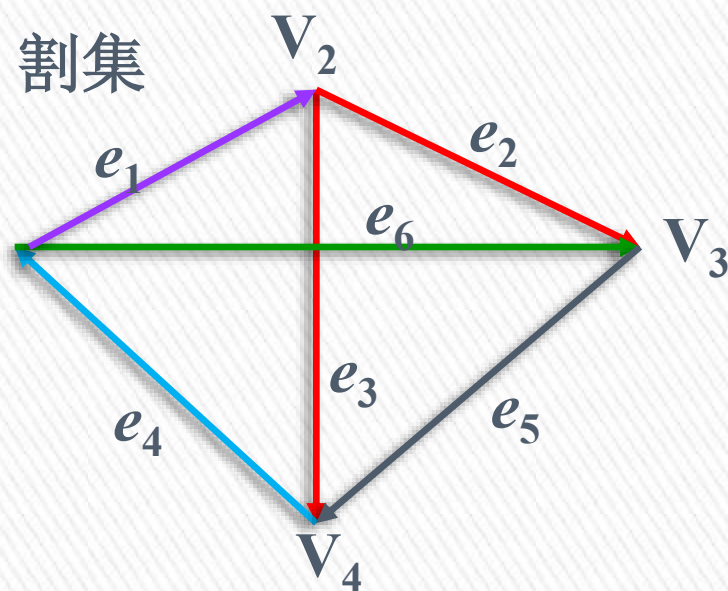
若 $b \notin S_e(t_1)$, 由于 $S_e(t_1)$ 不含 t_1 其他树枝边 $a_1, a_2, \dots, a_k$, 故 $S_e(t_1)$ 不含 t_2 的任何一条边。这样删去 $S_e(t_1)$ 的全部边之后, G 仍连通, 这与 $S_e(t_1)$ 是割集矛盾。

反之, 若 $b \in S_e(t_1)$, 而 $S_e(t_1)$ 只含 t_1 的唯一树枝 e 。因此 $b \neq a_i$ 。所以 $b \notin t_1$ 。 $t_1 + b$ 存在含 e 的唯一回路, 因此 $t_1 \oplus (e, b)$ 连通, 同时有 $n-1$ 条边。它是一棵树。

由于 $S_e(t_1)$ 中的边 b 是任意的, 所以立即可以进行推广。

支撑树的生成

定理3.7.2 给定 G 的一棵树 t_0 , 令 $t_1, t_2, \dots, t_p$ 是 G 中全部满足 $t_0 - t_i = (e)$, $t_i - t_0 = (b_i)$ 的树, 则 $S_e(t_0) = (e, b_1, b_2, \dots, b_p)$ 。反之, 若 $b_i \in S_e(t_0)$ 则 $t_i = t_0 \oplus (e, b_i)$ 是树



例3.5.2 图中 $t_0 = (e_1, e_2, e_3)$

则 $t_1 = (e_4, e_2, e_3)$,

$t_2 = (e_6, e_2, e_3)$

满足 $t_0 - t_1 = (e_1)$,

$t_1 - t_0 = (e_4)$,

$t_2 - t_0 = (e_6)$,

$S_{e_1}(t_0) = (e_1, e_4, e_6)$

支撑树的生成

例3.7.3

令图中 $t_0 = (e_1, e_2, e_3)$ 。每个树枝对应的割集是
 $S_{e_1}(t_0) = (e_1, e_4, e_6)$; $S_{e_2}(t_0) = (e_2, e_5, e_6)$; $S_{e_3}(t_0) = (e_3, e_4, e_5)$

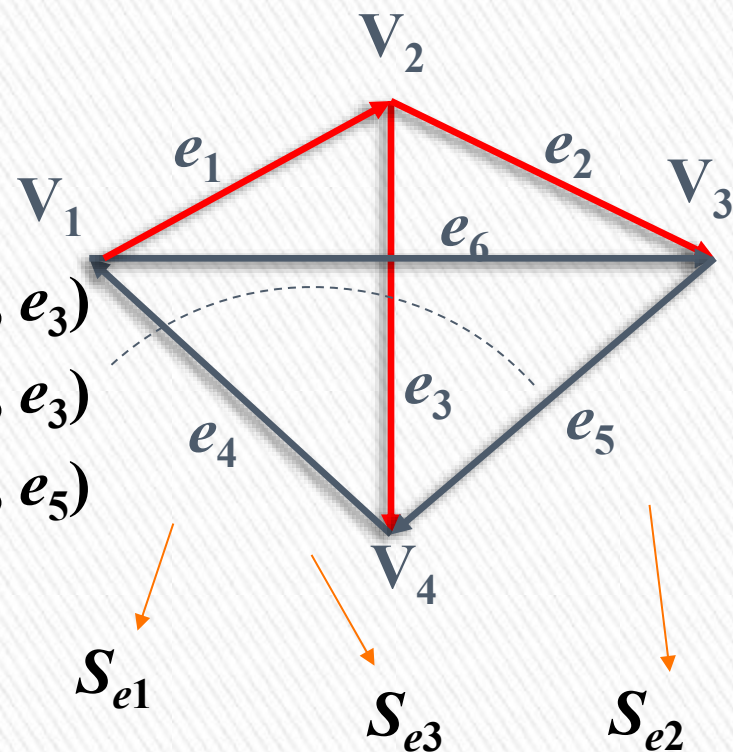
因此 G 中与 t_0 距离为1的全部树是

由 $S_{e_1}(t_0)$ 得: $t_1 = (e_4, e_2, e_3)$, $t_2 = (e_6, e_2, e_3)$

由 $S_{e_2}(t_0)$ 得: $t_3 = (e_1, e_5, e_3)$, $t_4 = (e_1, e_6, e_3)$

由 $S_{e_3}(t_0)$ 得: $t_5 = (e_1, e_2, e_4)$, $t_6 = (e_1, e_2, e_5)$

这样可以定义这些树的集合。



支撑树的生成

定义3.7.2 设 $T^e = \{t_0 \oplus (e, b) \mid b \in S_e(t_0), b \neq e\}$

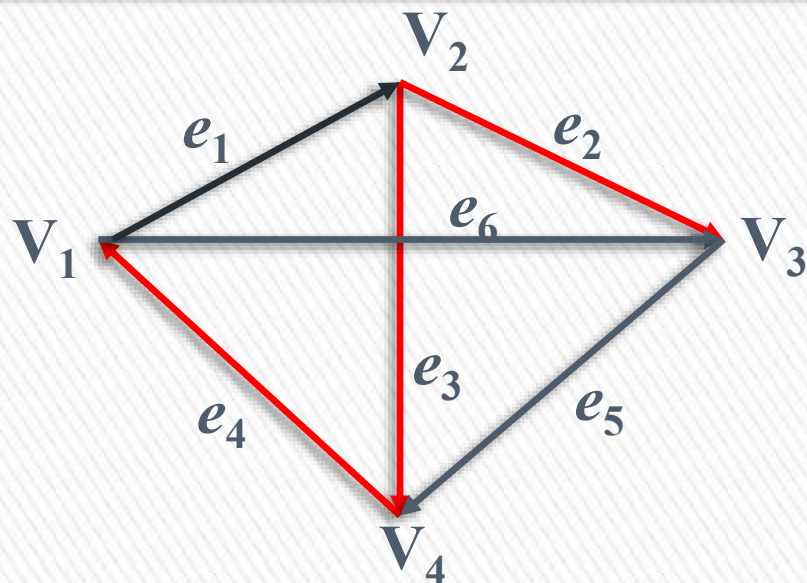
T^e 表示对 G 的某棵参考树 t_0 , $e \in t_0$ 。逐一用其基本割集 $S_e(t_0)$ 的每条边去替换 e , 即进行基本树变换之后的新树集合。

定理3.7.3 设 $t_0 = (e_1, e_2, \dots, e_k)$ 是 G 中的参考树。则 G 中与 t_0 距离为1的树恰在 $T^{e_1}, T^{e_2}, \dots, T^{e_k}$ 的某个集合之中。

定义 类似令 $T^{e_r e_s} = \{t \oplus (e_s, b) \mid b \in S_{e_s}(t), t \in T^{e_r}, b \neq e_s\}$

对参考树 t_0 中的一条树边 e_r , 先求替换 e_r 的与 t_0 距离为1的树的集合 T^{e_r} , 然后对 T^{e_r} 中的每棵树 t , 若 e_s 是它们的树枝, 再进行一次基本树变换。即使用 e_s 所对应的以 t 为树的基本割集中的其余边 b 来替换 e_s 。从而得到 $T^{e_s e_r}$ 中的一棵树。

支撑树的生成



$$t_1 = (e_4, e_2, e_3)$$

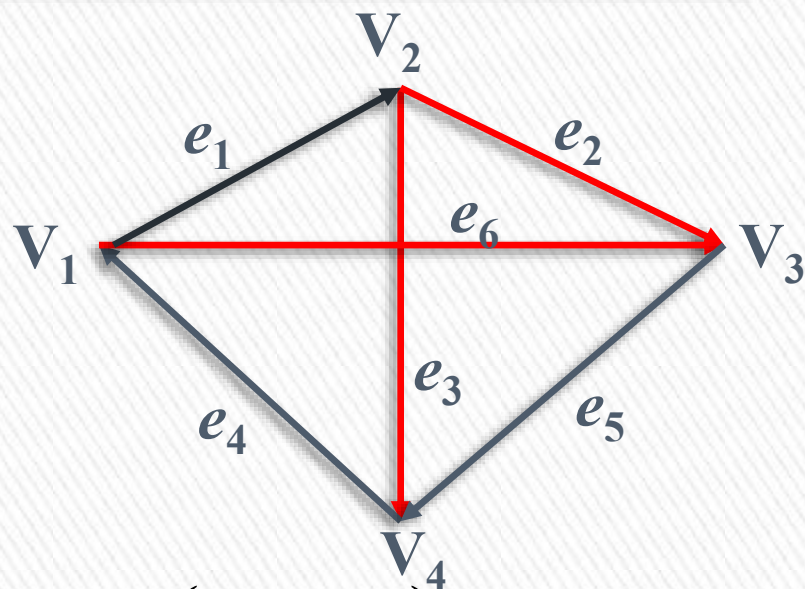
$$S_{e_2}(t_1) = (e_2, e_5, e_6)$$

由 $S_{e_2}(t_1)$ 得:

$$t_1' = (e_4, \textcolor{red}{e_5}, e_3)$$

$$t_2' = (e_4, \textcolor{red}{e_6}, e_3)$$

t_3' 就是例3.5.3中的 t_4 , 即它与 t_0 距离为1而其余均为2,
但 t_2', t_4' 是同一棵树



$$t_2 = (e_6, e_2, e_3)$$

$$S_{e_2}(t_2) = (e_2, e_1, e_4, e_5)$$

由 $S_{e_2}(t_2)$ 得:

$$t_3' = (e_6, \textcolor{red}{e_1}, e_3)$$

$$t_4' = (e_6, \textcolor{red}{e_4}, e_3)$$

$$t_5' = (e_6, \textcolor{red}{e_5}, e_3)$$

支撑树的生成

定义3.7.3 设 $t_0 = (e_1, e_2, \dots, e_{n-1})$ 是 G 的一棵参考树。

定义 $T^{e_1 e_2} = \{t \oplus (e_2, b) \mid b \in S_{e_2}(t) \cap S_{e_2}(t_0), t \in T^{e_1}, b \neq e_2\}$ 。

比如上例中

$$S_{e_2}(t_1) \cap S_{e_2}(t_0) = (e_2, e_5, e_6) \cap (e_2, e_5, e_6) = (e_2, e_5, e_6)$$

$$S_{e_2}(t_2) \cap S_{e_2}(t_0) = (e_2, e_1, e_4, e_5) \cap (e_2, e_5, e_6) = (e_2, e_5)$$

因此只有 t_1', t_2', t_3' 属于 $T^{e_1 e_2}$ 。由于 $S_{e_2}(t_0)$ 是图 G 关于树 t_0 的包含树枝 e_2 的基本割集。由于 e_1, e_2 都在 t_0 中，所以割集 $S_{e_2}(t_0)$ 不会包含另一条树枝 e_1 ，即

$e_1 \notin S_{e_2}(t) \cap S_{e_2}(t_0)$ ，当然 $e_2 \in S_{e_2}(t) \cap S_{e_2}(t_0)$ ，但 $b \neq e_2$ 。即 $T^{e_1 e_2}$ 中的每一棵树包含了 t_0 中除 e_1, e_2 之外的所有边，并对任意 $t \in T^{e_1 e_2}$ ，都有 $d(t, t_0) = 2$ 。因此可以得到结论。

支撑树的生成

定理3.7.4 设 $t_0 = (e_1, e_2, \dots, e_{n-1})$ 是距离为2的全部支撑树都在集合 $T^{e_i e_j}$ 之中, $e_i, e_j \in t_0, i \neq j$ 。

一般情况下可以定义 $T^{e_1 e_2 \dots e_k}$ 如下

$$\{t \oplus (e_k, b) \mid b \in S_{e_k}(t) \cap S_{e_k}(t_0), t \in T^{e_1 e_2 \dots e_{k-1}}, b \neq e_k, k < n\}$$

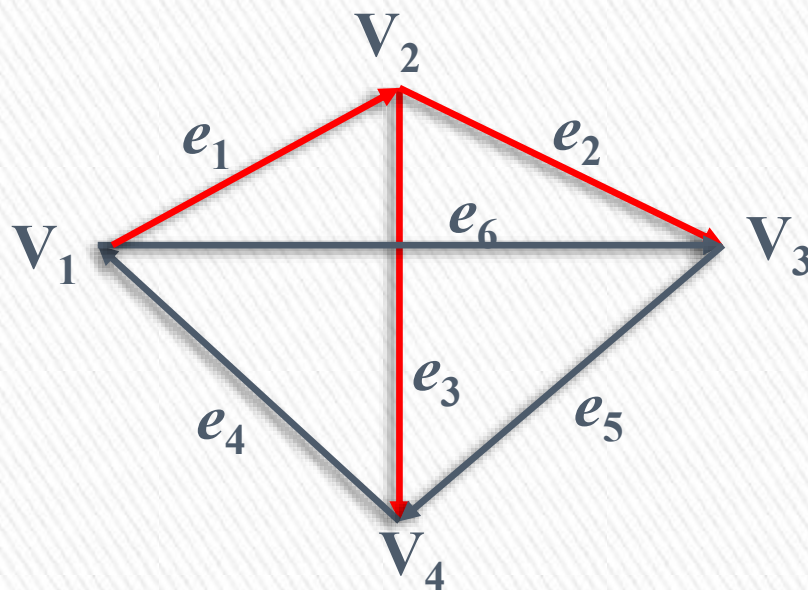
如果令 $T^1 = \cup T^{e_i}, e_i \in t_0$ 。 $T^2 = \cup T^{e_i e_j}, e_i, e_j \in t_0, \dots$ 。 $T^k = \cup T^{e_{i_1} e_{i_2} \dots e_{i_k}}, e_{ij} \in t_0, j = 1, 2, \dots, k$ 。

定理3.7.5 设 t_0 连通图 G 的参考树, $t \neq t_0$ 是 G 的任一棵支撑树, 那么一定有 $t \in \cup T^i$ 。

支撑树的生成

例3.7.5 以 $t_0 = (e_1, e_2, e_3)$ 为参考树，求 G 的全部生成树

1. $T^0 = \{t_0\} = \{(e_1, e_2, e_3)\}$

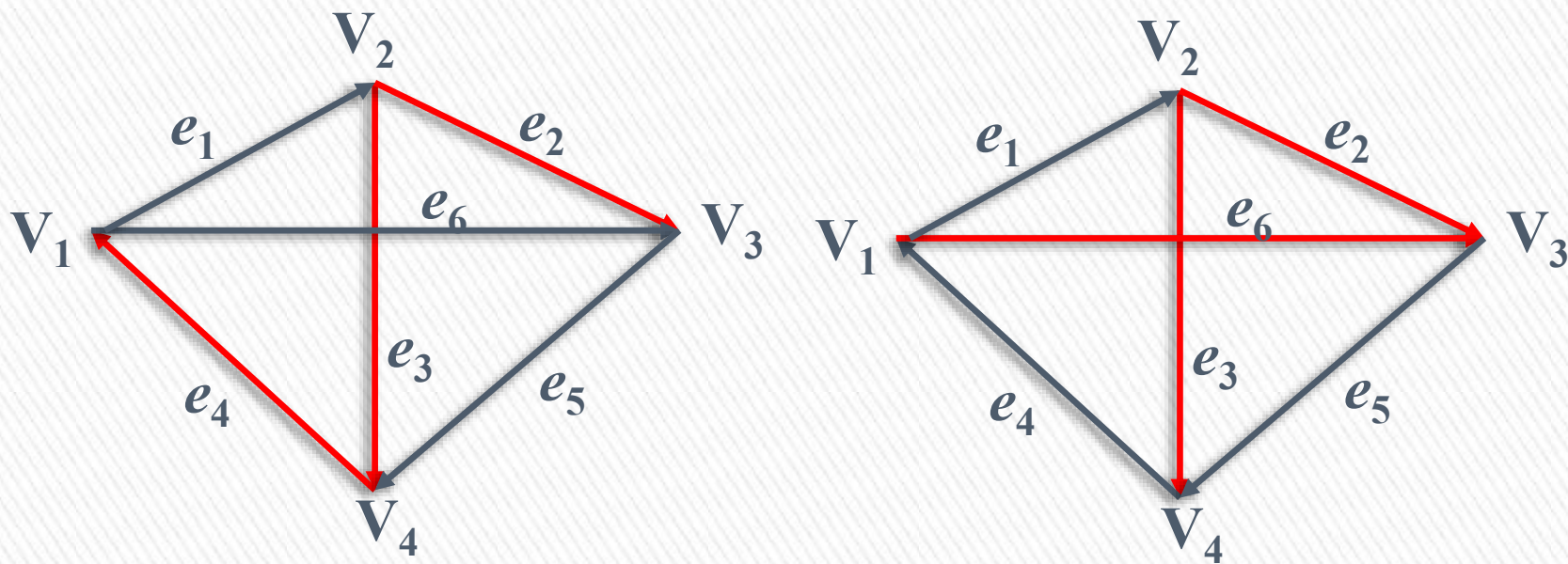


支撑树的生成

2. 生成距离为1的不同支撑树

$$T^1 = T^{e_1} \cup T^{e_2} \cup T^{e_3} = \{(e_4, e_2, e_3), (e_6, e_2, e_3)\} \cup \{(e_1, e_5, e_3), (e_1, e_6, e_3)\} \cup \{(e_1, e_2, e_4), (e_1, e_2, e_5)\}$$

■ 其中 $S_{e_1}(t_0) = (e_1, e_4, e_6)$, $S_{e_2}(t_0) = (e_2, e_5, e_6)$, $S_{e_3}(t_0) = (e_3, e_4, e_5)$

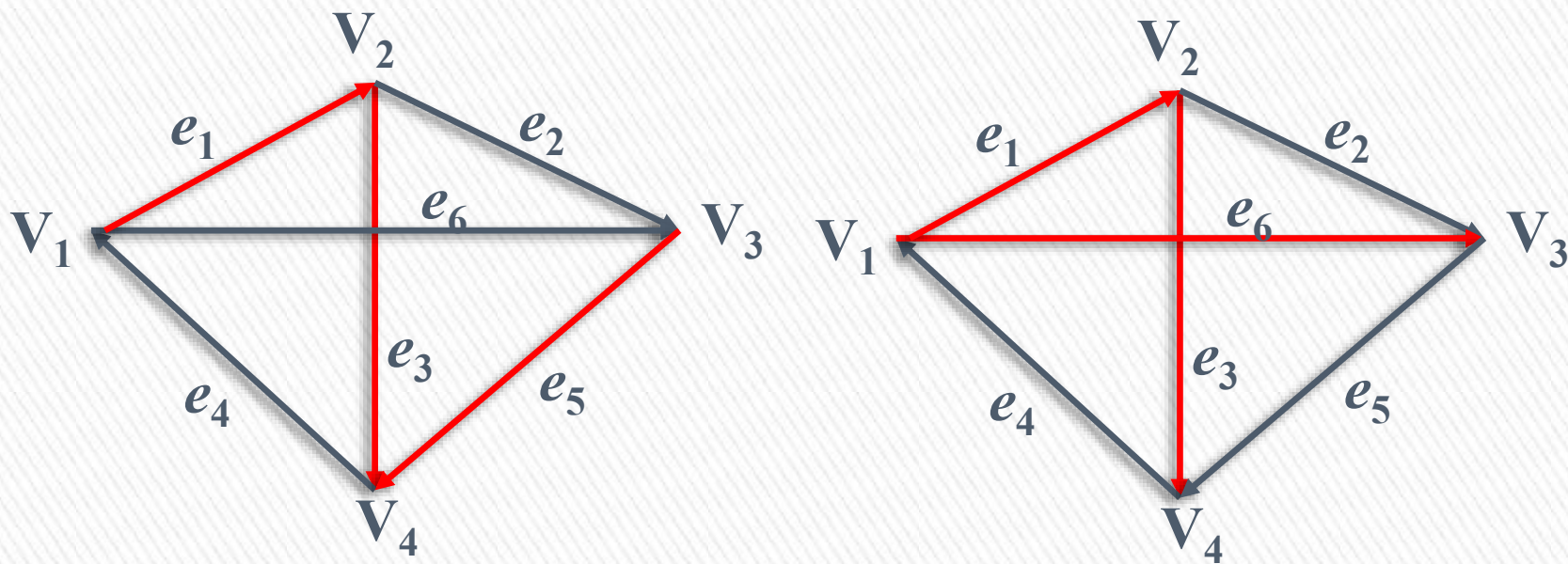


支撑树的生成

2.

$$T^1 = T^{e_1} \cup T^{e_2} \cup T^{e_3} = \{(e_4, e_2, e_3), (e_6, e_2, e_3)\} \cup \{(e_1, e_5, e_3), (e_1, e_6, e_3)\} \cup \{(e_1, e_2, e_4), (e_1, e_2, e_5)\}$$

■ 其中 $S_{e_1}(t_0) = (e_1, e_4, e_6)$, $S_{e_2}(t_0) = (e_2, e_5, e_6)$,
 $S_{e_3}(t_0) = (e_3, e_4, e_5)$

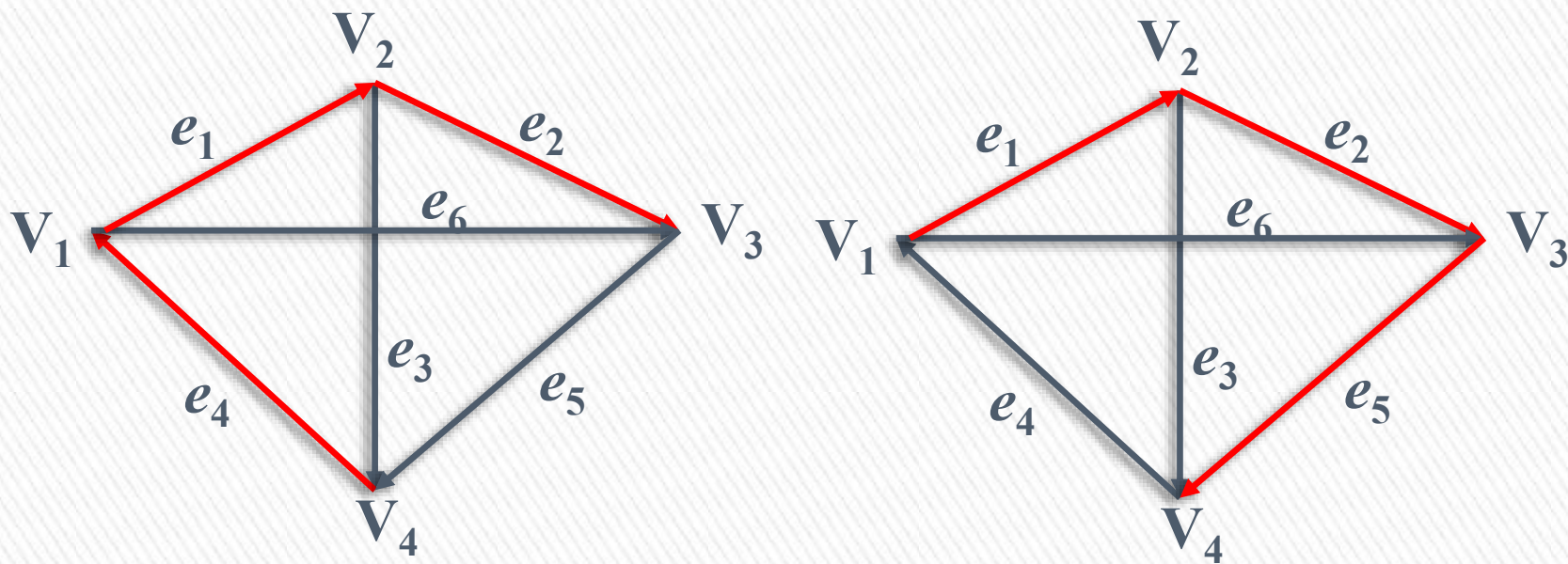


支撑树的生成

2.

$$T^1 = T^{e_1} \cup T^{e_2} \cup T^{e_3} = \{(e_4, e_2, e_3), (e_6, e_2, e_3)\} \cup \{(e_1, e_5, e_3), (e_1, e_6, e_3)\} \cup \{(e_1, e_2, e_4), (e_1, e_2, e_5)\}$$

■ 其中 $S_{e_1}(t_0) = (e_1, e_4, e_6)$, $S_{e_2}(t_0) = (e_2, e_5, e_6)$,
 $S_{e_3}(t_0) = (e_3, e_4, e_5)$



支撑树的生成

3. $T^2 = T^{e_1e_2} \cup T^{e_1e_3} \cup T^{e_2e_3}$

(1) $T^{e_1e_2} = \{(e_5, e_4, e_3), (e_6, e_4, e_3), (e_5, e_6, e_3)\}$

(2) 计算 $T^{e_1e_3}$ 。其中 T^{e_1} 包括 $t_1 = (e_4, e_2, e_3)$, $t_2 = (e_6, e_2, e_3)$
 $S_{e_3}(t_1) \cap S_{e_3}(t_0) = (e_1, e_3, e_5, e_6) \cap (e_3, e_4, e_5) = (e_3, e_5)$
 $S_{e_3}(t_2) \cap S_{e_3}(t_0) = (e_3, e_4, e_5) \cap (e_3, e_4, e_5) = (e_3, e_4, e_5)$

$T^{e_1e_3} = \{(e_5, e_4, e_2), (e_4, e_6, e_2), (e_5, e_6, e_2)\}$

(3) 计算 $T^{e_2e_3}$ 。其中 T^{e_2} 包括 $t_1 = (e_3, e_1, e_5)$, $t_2 = (e_3, e_1, e_6)$
 $S_{e_3}(t_1) \cap S_{e_3}(t_0) = (e_3, e_2, e_4, e_6) \cap (e_3, e_4, e_5) = (e_3, e_4)$
 $S_{e_3}(t_2) \cap S_{e_3}(t_0) = (e_3, e_4, e_5) \cap (e_3, e_4, e_5) = (e_3, e_4, e_5)$

$T^{e_2e_3} = \{(e_4, e_1, e_5), (e_4, e_1, e_6), (e_5, e_1, e_6)\}$

所以 T^2 包含了9棵树。

支撑树的生成

4. 计算 $T^3 = T^{e_1 e_2 e_3}$

由 $T^{e_1 e_2} = \{(e_5, e_4, e_3), (e_6, e_4, e_3), (e_5, e_6, e_3)\}$ 可得

$$S_{e_3}(t'_1) \cap S_{e_3}(t_0) = (e_1, e_2, e_3) \cap (e_3, e_4, e_5) = (e_3)$$

$$S_{e_3}(t'_2) \cap S_{e_3}(t_0) = (e_1, e_2, e_3) \cap (e_3, e_4, e_5) = (e_3)$$

$$S_{e_3}(t'_3) \cap S_{e_3}(t_0) = (e_1, e_2, e_3) \cap (e_3, e_4, e_5) = (e_3)$$

所以 $T^3 = \Phi$

支撑树的生成

T^0

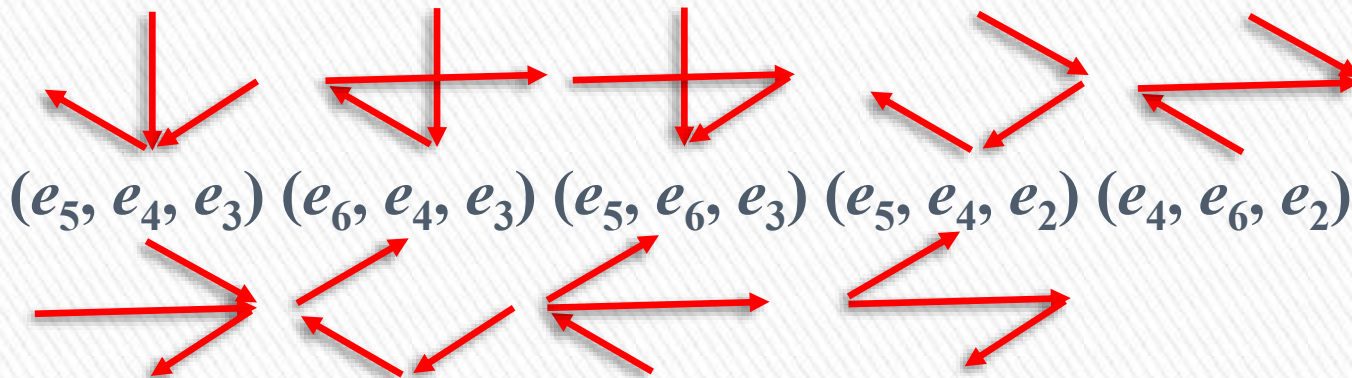


T^1



(e_4, e_2, e_3) (e_6, e_2, e_3) (e_1, e_5, e_3) (e_1, e_6, e_3) (e_1, e_2, e_4) (e_1, e_2, e_5)

T^2



(e_5, e_4, e_3) (e_6, e_4, e_3) (e_5, e_6, e_3) (e_5, e_4, e_2) (e_4, e_6, e_2)

(e_5, e_6, e_2) (e_4, e_1, e_5) (e_4, e_1, e_6) (e_5, e_1, e_6)

支撑树的生成

参考文献:

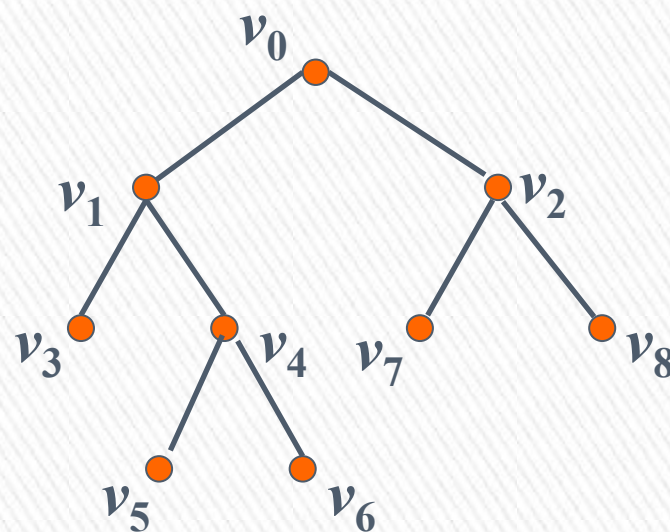
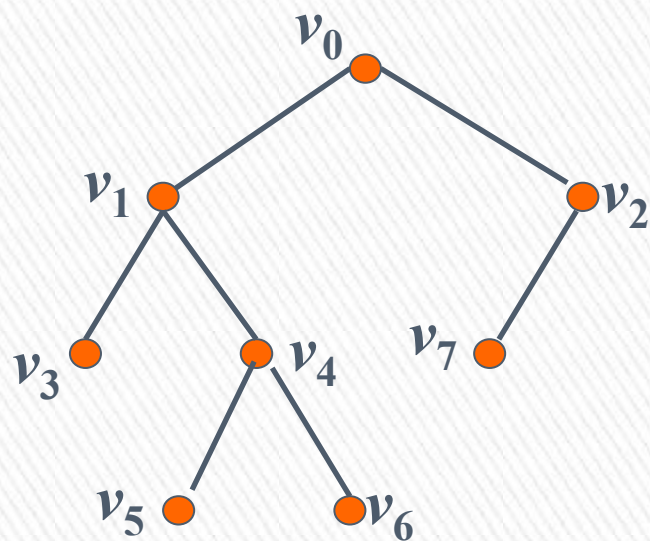
- [1] Mayeda W, Seshu S. Generation of trees without duplications[J]. IEEE Transactions on Circuit Theory, 1965, 12(2): 181-185.
- [2] Chakraborty M, Mehera R, Pal R K. A Divide-and-Conquer Algorithm for All Spanning Tree Generation[M]//Advanced Computing and Systems for Security. Springer, Singapore, 2017: 19-36.

Huffman树

二叉树

定义3.5.1 除树叶外，其余结点的正度最多为2的外向树称为**二叉树**

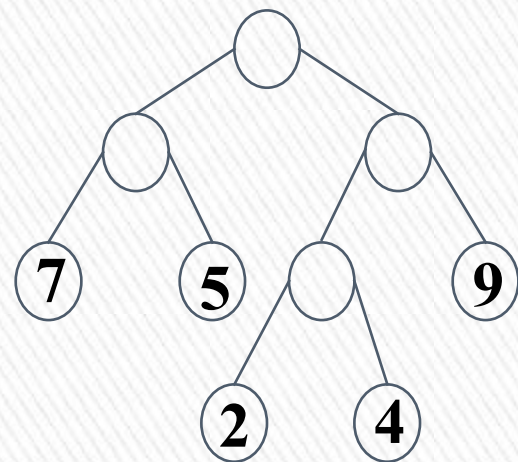
除树叶结点外，其余结点的正度都是2，称为**完全二叉树**



最优二叉树

定义3.5.2 给一棵完全二叉树的所有树叶都赋予权重 w_i ，称之为赋权二叉树，且将从根节点到树叶结点 $leafs\ v_i$ 的路径长度记作 l_i ，则该树的**带权路径长度**

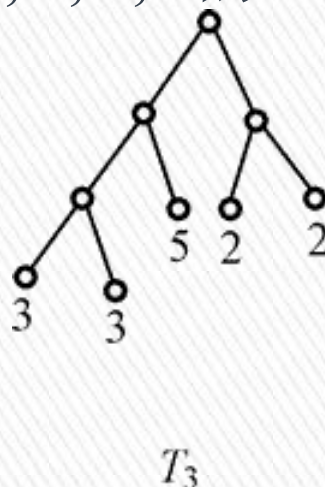
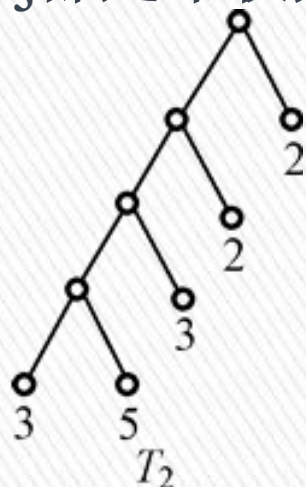
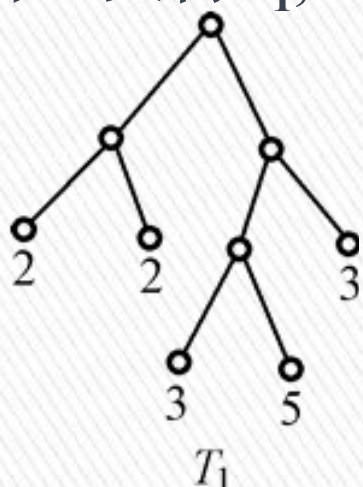
$$WPL = \sum_{i \in leafs} w_i l_i$$



定义3.5.3 在树叶结点数以及它们对应权值相同的情况下构造出的所有赋权二叉树中带权路径长度最短的树称为**最优二叉树**。

最优二叉树

下面三棵二叉树 T_1 , T_2 和 T_3 都是带权为2, 2, 3, 3, 5的二叉树



根据树权 $W(T)$ 的定义, 计算出它们的权分别为:

$$W(T_1) = 2*2 + 2*2 + 3*3 + 5*3 + 3*2 = 38$$

$$W(T_2) = 3*4 + 5*4 + 3*3 + 2*2 + 2*1 = 47$$

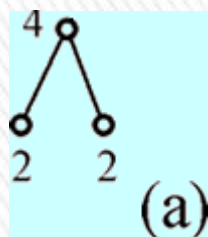
$$W(T_3) = 3*3 + 3*3 + 5*2 + 2*2 + 2*2 = 36$$

最优二叉树唯一吗? 如何构造?

Huffman树

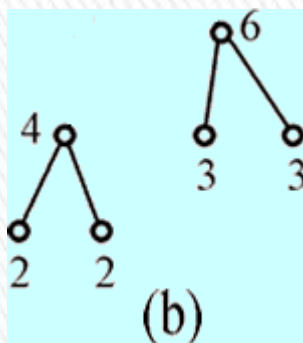
例：求带权2, 2, 3, 3, 5的最优二叉树

解：利用Huffman算法求解最优二叉树的全过程如下：



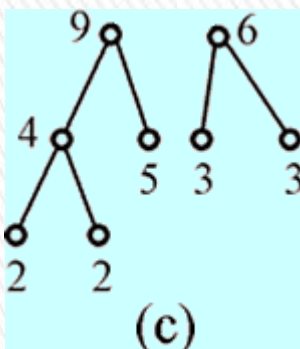
(a)

2, 2, 3, 3, 5



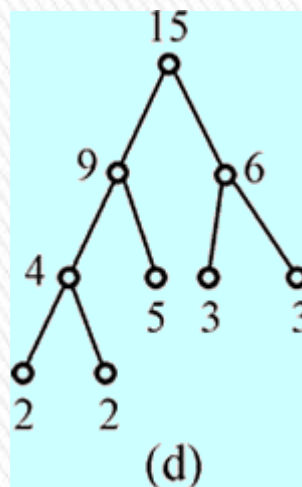
(b)

3, 3, 4, 5



(c)

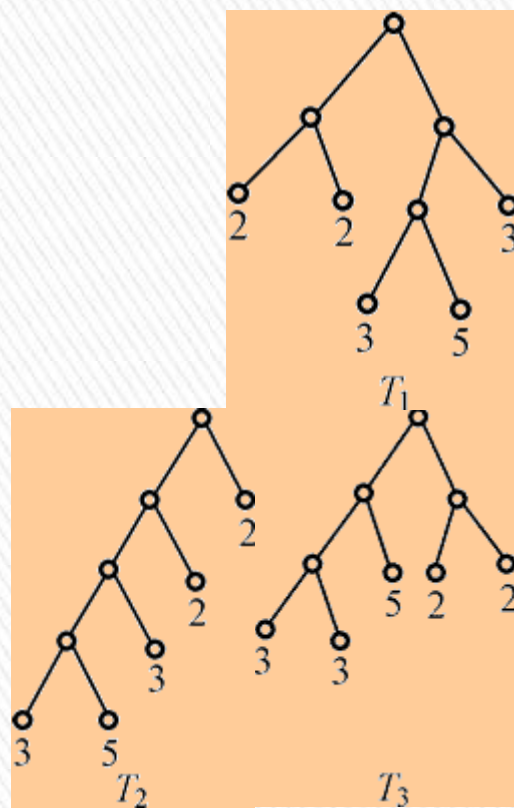
4, 5, 6



(d)

6, 9

显然，树(d)是最优树， $W(T) = 34$
右边三棵树都不是最优二叉树。



Huffman树

Huffman树构造算法

- 1、根据所有权重 $\{w_1, w_2, \dots, w_n\}$ 构造二叉树集合 $\{T_1, T_2, \dots, T_n\}$ ，其中树 T_i 仅有一个权重为 w_i 的结点；
- 2、将这些 w_i 按照从小到大进行排序，权重越小越靠前，放入优先级队列 Q 中；
 $O(n \log n)$
- 3、从 Q 中连续弹出两次队首元素并将对应的二叉树分别作为左右子树进行二叉树合并，合并后根结点权重=左子树根结点权重+右子树根结点权重，将该值根据大小插入优先级队列 Q 中；
 $O(\log n)$
- 4、重复步骤3，直到 Q 为空。
 $n - 1$ 次

整个算法计算复杂度 $O(n \log n)$

Huffman树

定理3.5.1 由Huffman算法得到的二叉树是最优二叉树。

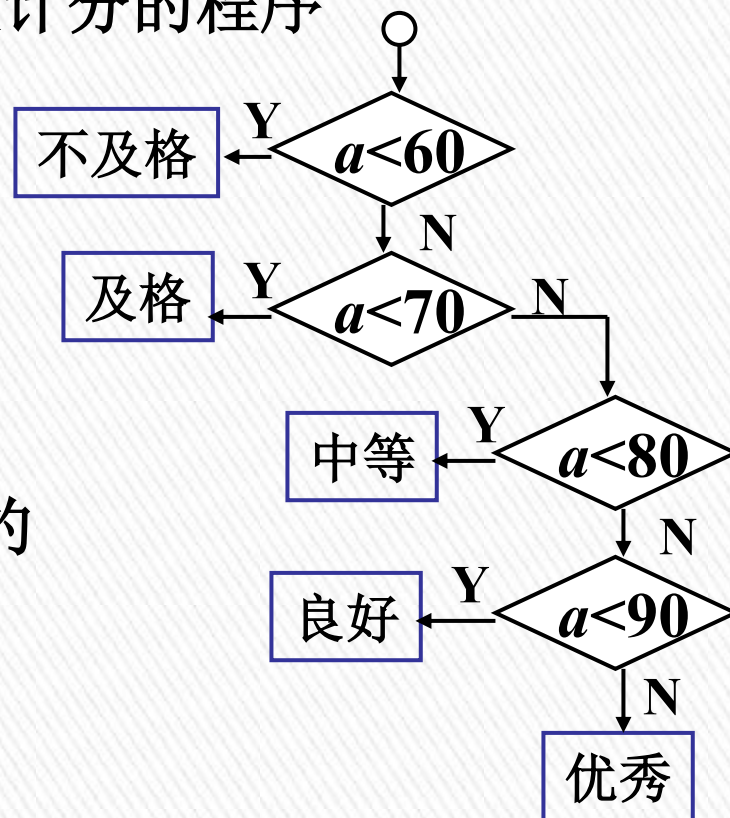
证明：

1. 对于最优二叉树 T ，假定 $n \geq 3$ ， $w_1 \leq w_2 \leq \dots \leq w_n$ ，一定有 $l_1 = \max\{l_i\}$ 。否则，如果存在 $w_k \geq w_1$ 且 $l_k \leq l_1$ ，那么交换 w_k 与 w_1 就能够得到权重更小的二叉树。
2. w_1 一定有兄弟结点，否则把 w_1 赋给其父节点，能够得到权重更小的二叉树。而且这个兄弟结点一定是 w_2 ，否则此树不是最优，道理同第一步。
3. 这样 w_1 和 w_2 以及他们的父节点构成一个最优子树，把它们合并为一个结点（权重相加）不改变树的最优性。
4. 重复这个合并的过程一直都不会改变树的最优性，直到三个结点为止，所以得到的树是最优二叉树。

Huffman树的应用

■ 用于程序设计

例：编写一个将百分分 a 转换成五计分的程序



这个算法是正确的，但不是最优的

Huffman树的应用

■ 用于程序设计

例：编写一个将百分分 a 转换成五计分的程序

在分数符合某种分布时，上述算法运行的时间不是最优的。
设分数的分布如下：

分数：	0-59	60-69	70-79	80-89	90-100
比例(%)：	5	15	40	30	10

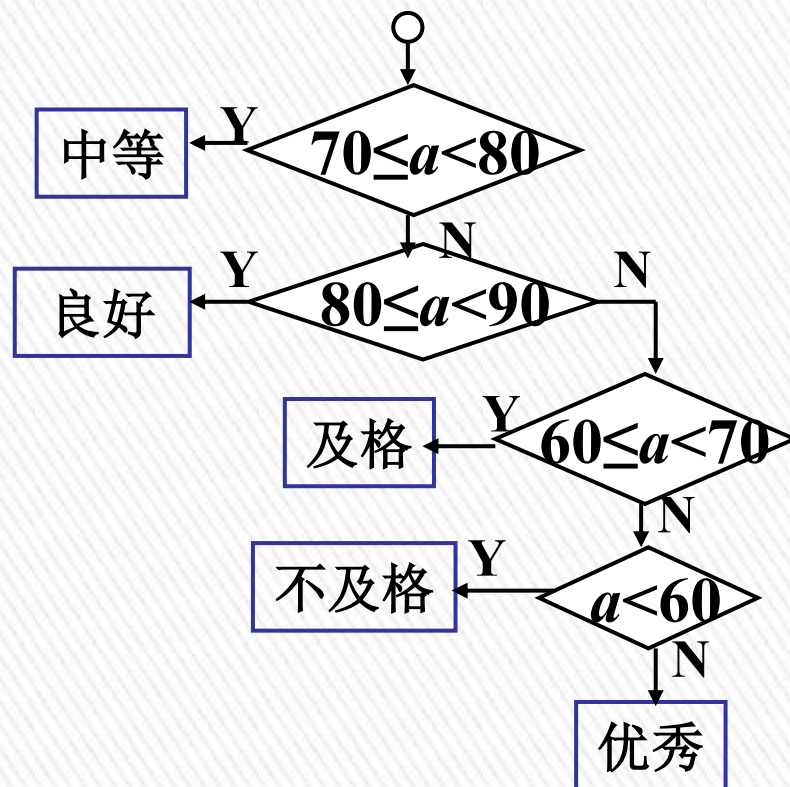
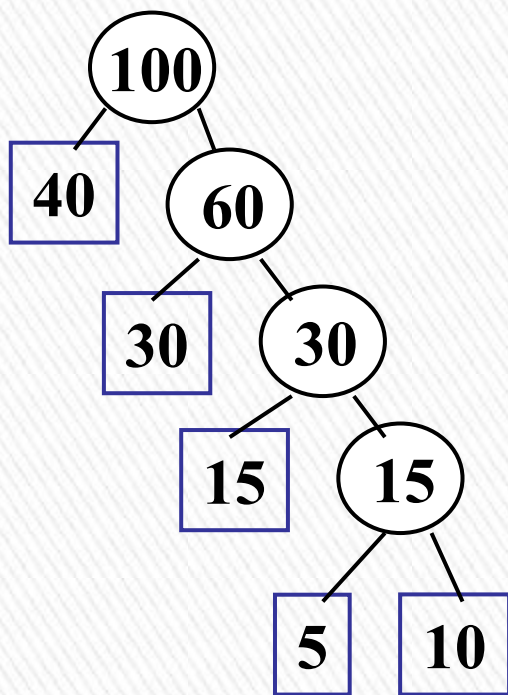
上述程序中，有80%的分数，至少要比对3次(因为80%的分数>70分)才得出结果。那么如何设计这个程序才合理呢？就是按最优树来设计。

Huffman树的应用

分数: 0-59 60-69 70-79 80-89 90-100

比例(%): 5 15 40 30 10

设权序列为: 5, 10, 15, 30, 40 构造最优树:



Huffman树的应用

■ 应用场景：数据压缩表示

考虑在计算机中存储一段数据，其中A, B, C, D出现个数分别为5, 7, 2, 13

naïve方法：

每个字符 → 2位二进制编码 (00, 01, 10, 11)

原数据占用空间 → $2 \times 27 = 54$ 位

■ 问题：在保证字符串数据没有损失的情况下，如何编码使存储字符串所需空间尽量小？

出现频率高的符号用位数少的编码来表示

Huffman树的应用

■ 编码

定义 设 $a_1a_2\dots a_{n-1}a_n$ 是长度为 n 的符号串，称其子串 $a_1, a_1a_2, \dots, a_1a_2\dots a_{n-1}$ 分别为该符号串的长度为 $1, 2, \dots, n-1$ 的**前缀**；

设 $A = \{b_1, b_2, \dots, b_m\}$ 为一个符号串集合，若对于 $\forall b_i, b_j \in A, i \neq j$, b_i 和 b_j 互不为前缀，则称 A 为前缀码；

若符号串 $b_i (i = 1\dots m)$ 中只出现0和1，则称 A 为二元前缀码；

Huffman树的应用

■ 前缀码

例如：

{1, 00, 011, 0101, 01001, 01000}为前缀码，

{1, 00, 011, 0101, 0100, 01001, 01000}不是前缀码，因为0100既是01001又是01000的前缀。

可用二叉树产生二元前缀码。

设 T 是具有 k 个叶结点的二叉树， v 为 T 的内点：

- 若 v 只有一个孩子，则在连接 v 的边上可标0或1；
- 若 v 有两个孩子，在连接 v 的左边上标0，右边上标1；

Huffman树的应用

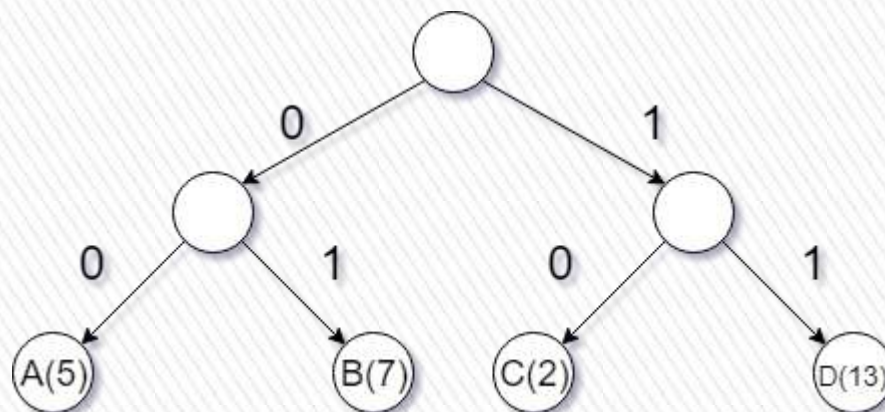
■ 前缀码

设 v_i 是 T 的一个叶结点，从树根到 v_i 的通路上各边的标号(0或1)，按通路上边的顺序组成的符号串放在 v_i 处，则 k 个叶结点处 k 个符号串组成的集合为二元前缀码。

因为根到树叶的路径是唯一的，而且这条路径绝不会是根到另一个树叶路径的一部分，因此，所得符号串集合必为前缀码。

Huffman树的应用

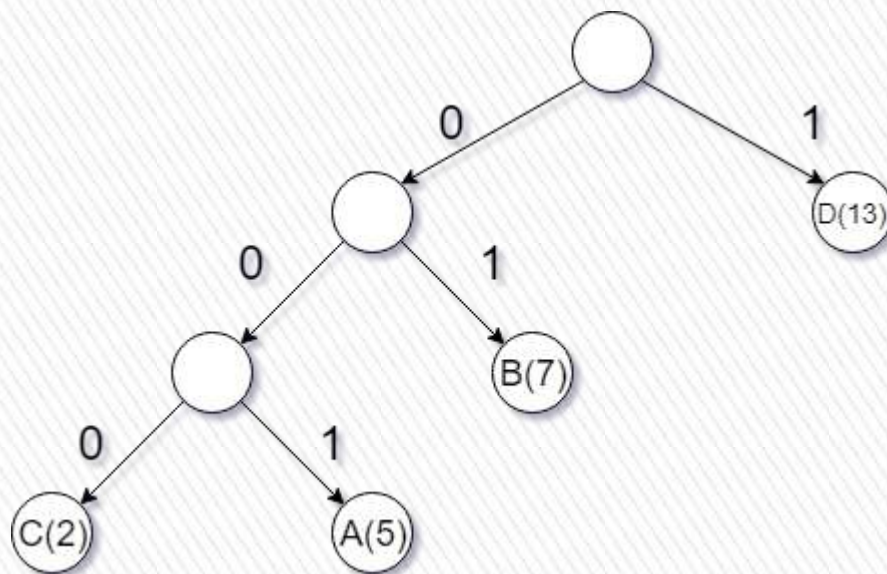
■ naïve编码方式:



$$WPL = 2 \times 5 + 2 \times 7 + 2 \times 2 + 2 \times 13 = 54$$

Huffman树

■ 考虑另一种编码方式:



$$WPL = 3 \times 2 + 3 \times 5 + 2 \times 7 + 1 \times 13 = 48$$

能够节省存储空间!

Huffman树的应用

二叉树产生出来的前缀码都可无歧义的传输多个符号，但当这些字母出现频率不同时，哪一个符号串传输哪个字母最省呢？

我们可用“符号出现的频率”为权，用Huffman算法求最优二叉树，由最优二叉树产生的前缀码称为最佳前缀码，用最佳前缀码传输对应的符号可使传输的二进制数位最省。

Huffman树的应用

例：在通信中，八进制数字出现的频率如下：

0: 25% 1: 20% 2: 15% 3: 10%

4: 10% 5: 10% 6: 5% 7: 5%

求传输它们的最佳前缀码，并求传输 $10^n (n \geq 2)$ 个按上述比例出现的八进制数字需要多少个二进制数字？若使用等长的(长为3)的码传输需要多少个二进制数字？

由题意可知：8个权(乘100后)为 $w_7 = 5, w_6 = 5, w_5 = 10, w_4 = 10, w_3 = 10, w_2 = 15, w_1 = 20, w_0 = 25$ 。

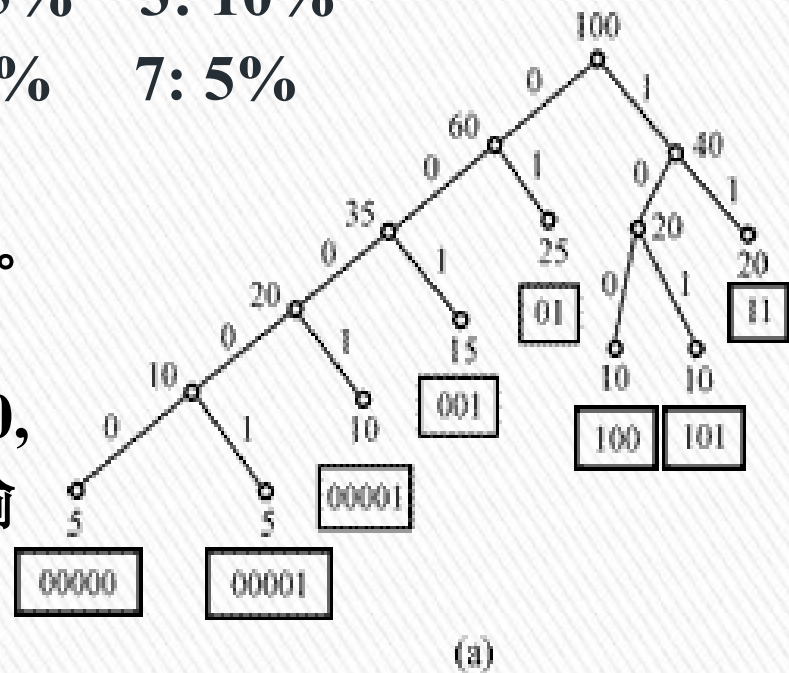
Huffman树的应用

例：在通信中，八进制数字出现的频率如下：

0: 25% 1: 20% 2: 15% 3: 10%
4: 10% 5: 10% 6: 5% 7: 5%

解（续）

用Huffman算法求最优树如右图(a)。图中矩形框中的符号串为对应数字的编码，即可用编码01, 11, 001, 100, 101, 0001, 00000和00001来分别传输0, 1, 2, 3, 4, 5, 6和7。



8个编码的集合{01, 11, 001, 100, 101, 0001, 00000, 00001}为前缀码，且是最佳前缀码。

Huffman树的应用

设图(a)的树为 T ，显然， $W(T)$ 为传输100个题中八进制数字所用二进制数字个数。

$$W(T) = 285。$$

传输 10^n 个题中八进制数字需要 $10^{n-2} \times 285 = 2.85 \times 10^n$ 个二进制位。

用长度为3的0/1符号串传输 10^n 个八进制数字(如: 000传0, 001传1, ..., 111传7)，所以，需用 3×10^n 个二进制位。

由此可见：用前缀码进行编码，可提高传输效率。

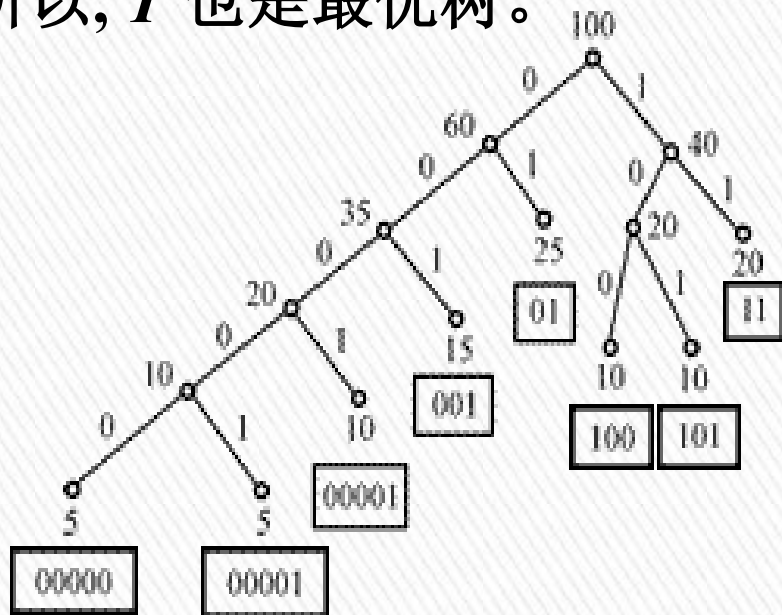
Huffman树的应用

最佳前缀码唯一吗？

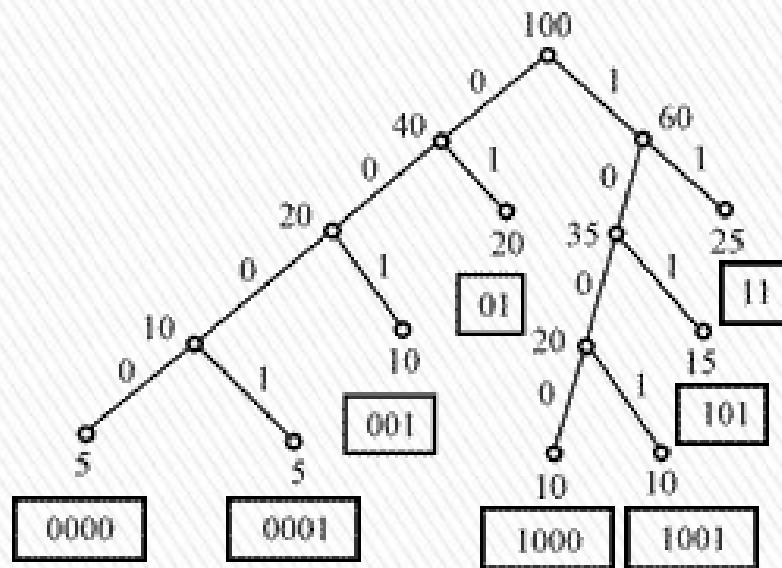
不是唯一的。因为选择两个最小权的选法不唯一，和两个权所对应的顶点放在左右位置也可不同，但它们的带权路径长度都是相等的，即它们都是最优树。

$$W(T') = 10 + 20 + 40 + 100 + 60 + 35 + 20 = 285 = W(T).$$

所以, T' 也是最优树。



(a)



(b)